Applied Intelligence (2022) 52:7136–7157
https://doi.org/10.1007/s10489-021-02505-0

Mining interesting sequences with low average cost and high average utility

Tin Truong[1] · Hai Duong[1] · Bac Le[2,3] · Philippe Fournier-Viger[4] · Unil Yun[5]

Accepted: 4 May 2021 / Published online: 20 September 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract
Discovering high utility sequences in a quantitative database is a popular data mining task. The goal is to enumerate all sequences of items (symbols) that have a high value for the user, as measured by a utility function. A representative application of high utility sequence mining is the identification of profitable sequences of purchases in transactions from online stores. Though useful, a drawback of that task is that the cost of items is not considered. However, cost is a key factor for decision-making in that domain and many others. To consider both the cost and utility of items for sequence mining, this paper defines a novel problem *FLCHUSM* of mining frequent sequences having a high average utility and a low average cost. Though the proposed problem is a generalization of the traditional problem of frequent sequence mining, it is more challenging because the average utility and average cost functions do not satisfy the downward-closure property traditionally used to reduce the search space. To offer a solution to this issue, this paper presents a lower bound on the cost and two novel upper bounds on the utility. Besides, four width, depth pruning, reducing and tightening strategies are devised to eliminate unpromising patterns from the search space. Taking these theoretical results as a foundation, a new *CUL* (*Cost-Utility List*) data structure is conceived for storing and quickly updating the utility and cost information of patterns, and a novel algorithm named FLCHUSPM is proposed for *FLCHUSM*. Results from several experiments show that FLCHUSPM is efficient in terms of memory usage and runtime, and that interesting patterns can be discovered in real data.

Keywords Quantitative sequence database · Average utility · Average cost · Upper bound · Lower bound · Pruning strategy

1 Introduction

Frequent pattern mining (FPM) is a well-studied knowledge discovery problem. The aim is to find interesting patterns in (transaction or sequence) databases (DBs), where a pattern is viewed as interesting if it has a high occurrence frequency (also called support) [1–3]. The rationale for selecting patterns based on their frequencies is that frequent patterns describe the most common situations occurring in the data. Nonetheless, the support function is not always the most relevant one as rare

✉ Philippe Fournier-Viger
philfv@hit.edu.cn

Tin Truong
tintc@dlu.edu.vn

Hai Duong
haidv@dlu.edu.vn

Bac Le
lhbac@fit.hcmus.edu.vn

Unil Yun
yunei@sejong.ac.kr

[1] Department of Mathematics and Computer Science, Dalat University, Dalat, Vietnam

[2] Deparment of Computer Science, University of Science, Ho Chi Minh City, Vietnam

[3] Vietnam National University, Ho Chi Minh City, Vietnam

[4] School of Humanities and Social Sciences, Harbin Institute of Technology (Shenzhen), Shenzhen, China

[5] Department of Computer Engineering, Sejong University, Seoul, South Korea

pattern may be more important than frequent ones when considering other factors. For instance, in an e-learning context, sequences of actions made by learners that lead to the highest scores or display the best learning efficiency may not be the most frequent ones. Similarly, in the context of online sale analysis, the total profit generated by a pattern (a set of purchased products) may be viewed as more important than its sale frequency, as several products yield a low profit.

Based on this observation, several other criteria for pattern selection have been employed in pattern mining. A popular generalization of the frequent pattern mining task, called high utility pattern mining, evaluates a pattern's importance using a numeric utility function. A pattern's utility can represent various desirable factors such as the total score obtained by a student or the profit yield by product sales. High utility patterns are discovered in so-called quantitative DBs, where each item is associated with an internal and an external utility value (positive numbers). These values can model various aspects such as the number of items purchased by a customer and each item's unit profit, respectively. Several utility functions have been defined to evaluate a pattern's utility, including the traditional support measure used in frequent pattern mining. A pattern is called a high utility (HU) pattern if its utility is equal to or greater than a user-defined minimum utility threshold, denoted as *mu*. During the last decade, the problem of enumerating all HU patterns in a database has attracted the interest of numerous researchers as it has applications in many fields such as for analyzing web logs [4, 5], mobile commerce data [6], gene regulation data [7], hospital event logs [8], and noisy environments [9].

Multiple variations of the HU pattern mining problem have been considered. In high utility itemset mining (HUIM), a search for itemsets (sets of items) of high utility is done in a quantitative transaction database (QTDB) [10, 11]. Each itemset has a utility value, computed as the sum of its utility values in each transaction that contains it. This problem can be instantiated for applications such as finding all sets of profitable purchases in customer transactions. But because HUIM does not consider the order between items (e.g. the time ordering between purchases), a more general problem of high utility sequence mining (HUSM) was introduced. The input data is a quantitative sequence database (QSDB), where itemsets are sequentially ordered, items have internal/external utility values, and a sequence may appear multiple times in each input quantitative sequence. Given such data, HUSM is the task of enumerating all high utility sequences, that is subsequences of the input database having a high utility (e.g. that produce high scores or are profitable). That task is harder than HUIM because a sequence may appear several times in each database record (input sequence). Consequently, there are numerous ways of calculating the utility of a sequence and the calculation can be more time consuming than in HUIM.

The problems of HUSM and HUIM are also more difficult than FPM as the famous downward-closure property of the support in FPM (also known as the anti-monotonic - $\mathcal{AM}$ - property) is not satisfied by most utility functions. The downward-closure property plays a major role in reducing the search space in FPM. For a minimum support threshold *ms*, the $\mathcal{AM}$ property indicates that if a pattern is infrequent, all its super-patterns can be ignored from the search space since they are also infrequent. In HUSM, a utility function u is generally neither anti-monotonic nor monotonic. To deal with this key challenge, an upper bound (UB) on u, named *SWU* (*Sequence-Weighted Utility*) [12], which respects $\mathcal{AM}$, or tighter UBs on u that satisfy anti-monotonic-like properties that can be weaker than $\mathcal{AM}$ have been proposed and integrated into efficient HUSM algorithms [13]. Some researchers have also studied the problem of finding compact representations of sequences such as generator and closed patterns [14–18].

While there are many applications for HUIM and HUSM, a major issue is that extensions of high utility patterns (HUPs) with frequent items are often also HUPs. Hence, the set of all HUPs is sometimes enormous, and thus difficult for users to browse through and understand. In addition, the utility of patterns tends to increase with their length, which may be perceived as unfair. To overcome this drawback, researchers introduced the *average utility* function to evaluate a pattern, which is calculated as its utility divided by its length [19–23]. This measure is often deemed as fairer than other utility measures as it considers pattern length, and to be more useful for decision-making [20, 21, 24].

Nevertheless, a drawback of most studies on HUP mining is that they have disregarded another important aspect, that is each pattern's cost. In some contexts, it is essential to not only measure the average utility of patterns but also their average cost. This is motivated by a practical example. Consider an e-learning application where a special type of QSDBs is recorded, and each input sequence is a series of activities performed by a student. In such database, a student activity may be associated with not only a *utility* (representing the result obtained by performing that activity such as a grade) but also a cost (e.g. time spent on the activity). Then, an interesting problem, which will be studied in this paper, is to extract the frequent activity sequences performed by several students that have a *low average cost* and result in a *high average utility* (per student). Such patterns are interesting to understand learning behavior as they may indicate popular ways of learning that have a low average cost but resulted in high average grades. The concept of cost and utility can also be found in several other applications. For instance, sequences of tourist attractions visited by tourists in a city can be stored in a mobility sequence database. In a user sequence, each tourist attraction may be annotated with a cost representing the money spent at that tourist attraction (e.g. entrance fee and the purchase of

souvenirs) and a utility value representing the customer satisfaction. Then, sequences with low average cost and high average utility may be extracted to recommend itineraries that have a low average cost and yield a high satisfaction to tourists. In the context of this paper, the *average* utility *au* (or *average* cost *ac*) of a sequence is computed as its utility (or cost) divided by the number of input sequences containing the pattern. To solve this problem of mining all frequent sequences with low average cost and high average utility (a problem called $\mathcal{FLCHUSM}$), a key challenge is that the *au* and *ac* measures do not satisfy anti-monotonic and monotonic properties, respectively. Thus, devising upper bounds (UBs) on *au* and lower bounds (LBs) on *ac* that respect anti-monotonic and monotonic properties is very important to be able to reduce the search space. Moreover, another challenge is that calculating the *au* and *ac* of sequences can be much more time-consuming than calculating the *support*.

This paper addresses these challenges by studying a novel problem named *FLCHUSM* (*frequent low average cost* and *high average utility* sequence mining), which aims at finding *frequent low cost* and *high utility* (FLCHU) sequences. The three main contributions of this article are:

(1) Formalizing the *FLCHUSM* problem.
(2) Designing UBs on *au* and LBs on *ac* that satisfy anti-monotone-like and monotone-like properties, respectively, and corresponding pruning strategies to eliminate unpromising candidates early.
(3) Proposing a suitable data structure and integrating it into a novel algorithm named FLCHUSPM for efficiently and completely mining all FLCHU sequences.

Results from the experiments presented in this study show that the designed FLCHUSPM algorithm is efficient, benefits from the pruning strategy, has good scalability and can discover interesting patterns in real e-learning data.

The next sections contain the following content. Section 2 describes related work. Section 3 defines a generic framework for the *FLCHUSM* problem. Section 4 proposes two new UBs on the average utility, an LB on the average cost, as well as strategies for pruning the search space. Thereafter, Section 5 introduces the new FLCHUSPM algorithm. To assess the algorithm performance, results from an experimental study conducted on both synthetic and real-life QSDBs are presented and discussed in Section 6. Finally, Section 7 wraps up the paper with the conclusion and a discussion of future work.

2 Literature review

Most studies on finding interesting patterns have targeted discovering frequent and/or high utility patterns. As explained by Yin et al. [12], the utility function is more general than the support function of FPM. The most common utility function for HUIM in QTDBs is defined as a summation [11, 25, 26]. But since the lengths of patterns tend to be correlated with their utility, the set of HUPs can be huge, and users may face difficulties in analyzing or comprehending all HUPs. For this reason, an alternative utility function named *average utility* was defined, calculated as the ratio of a pattern's utility to its length [19]. Because the length is taken into consideration, this function is generally deemed fairer than the traditional summation-based utility. It also has applications in various fields. To solve the problem of high average utility itemset mining (HAUIM) efficiently, algorithms were designed, which use an anti-monotonic upper bound on the average utility named MU (*Maximum Utility*) [20]. The MU upper bound of an itemset is calculated as the maximum utility among all utilities of transactions containing the itemset. Search space reduction is done using the MU by identifying sequences that have a low utility early during the pattern mining process. Nonetheless, the effectiveness of MU-based search space pruning is sometimes low as the MU is a loose upper bound on the average utility. To be more effective at reducing the search space for HAUIM, upper bounds that are tighter than the MU have been found [21, 22, 24, 27].

Different from HUIM or HAUIM where a pattern can occur at most once per transaction, a pattern may occur multiple times per input sequence of a QSDB in HUSM. Consequently, there are many ways of calculating a pattern's utility. Two popular utility functions are defined using a maximum form and a minimum form, respectively, and are denoted as u_{max} and u_{min} [28, 29]. Since the $\mathcal{AM}$ property does not hold for u_{max} and u_{min}, an upper bound (UB) on the two utility measures satisfying $\mathcal{AM}$, named SWU (*Sequence Weighted Utility*), was proposed to reduce the search space. The SWU UB of a sequential pattern is the utility sum of all items in input sequences containing the pattern. Nevertheless, because the SWU UB is too loose, other tighter UBs on u_{max} (CROM in [30], PEU and RSU in [31], SEU and PEU in [13, 32–34]) and on u_{min} (RBU and LRU in [14, 29]) were proposed. It was demonstrated that these upper bounds can be used to identify unpromising candidate sequences early to improve mining efficiency. It is interesting to note that the SPU measure proposed in [12, 35] is not a true UB on u_{max} as reported in [36]. In other words, algorithms using the SPU as UB on u_{max} for mining high utility sequences can be incomplete.

Inspired by the concept of high average utility and its fairer measurement of the utility of patterns for HAUIM, the problem of HUSM was redefined as high average utility sequence mining (HAUSM) [37]. In that problem, the average utility of a sequence is calculated as its utility over its length. But a limitation of that definition is that the average utility function is only defined for the special case of a QSDB containing each

item no more than once per input quantitative sequence. This means that a subsequence cannot appear more than once in each input sequence, which is an unrealistic assumption for some applications. For instance, in market basket analysis, a customer may purchase the same products more than once. Thilagu and Nadarajan [37] designed a simple upper-bound on that average utility function, named *atwu* (*average transaction weighted utility*). In more details, the *atwu* UB of a sequence α can be calculated as the total utility of transactions containing α divided by its length. However, algorithms using the *atwu* cannot reduce much of the search space as it is a quite large upper bound on the average utility. In recent years, some tighter upper bounds were designed [23, 38] for HAUSM.

Though problems of mining high (average) utility patterns have many applications, a drawback is that the cost of each pattern is disregarded. However, the cost is a critical aspect of many applications. To address this issue, this paper studies the problem of mining frequent sequences having a high average utility and low average cost in general QSDBs where each item of any input sequence is associated with both utility and cost values. Moreover, the average utility of a sequential pattern α is defined differently than in previous work. That is the average utility of α is defined as its utility divided by its support instead of its length. Thus, techniques that are used in previous work to extract patterns and reduce the search space cannot be directly applied to the proposed problem.

Note that two recent studies [39, 40] may appear to be close to the problem presented in this paper as they have also proposed algorithms for discovering sequential patterns in a sequence database, where each item is associated with a cost value. However, these studies differ from the current study in three major ways. First, these studies only associate a utility value to each input sequence rather than to each input sequence item. As a result, these models are suitable for applications that are different than those considered in this study. Second, utility calculations are simpler, which can lead to results that may be considered unusual such that two distinct sequential patterns have the same utility in the same input sequence. Third, a limitation of these two studies is that the cost of a pattern is only calculated based on its first (or last) occurrence in input sequences rather than by considering all occurrences.

The next section describes the proposed problem of mining frequent, low average cost and high average utility sequences in general QSDBs where each item in an input sequence is associated with both utility and cost values, and the average utility *au* and average cost *ac* of a sequential pattern are defined in a more general way than the two aforementioned studies. Then, the following sections present novel upper bounds on *au* and lower bounds on *ac*, the proposed algorithm, and the experimental evaluation.

3 Problem definition

This section presents the problem of frequent, low average cost and high average utility sequence mining (*FLCHUSM*) in an arbitrary quantitative sequence database. Let $\mathcal{A} = \{a_1, a_2, \ldots, a_M\}$ represent a finite set of m distinct items appearing in a database. A subset E of $\mathcal{A}$, $E \subseteq \mathcal{A}$, is said to be an *itemset*. It is assumed that the items within an itemset are arranged based on the lexicographical order $\prec$. Let $\alpha = E_1 \rightarrow E_2 \rightarrow \ldots \rightarrow E_p$ denote a *sequence* that is a list of itemsets E_k, $k=1, \ldots, p$. In a quantitative database (with cost and utility values), each item $a \in \mathcal{A}$ is associated with a utility value *ut* and a *cost* value *co* indicating the cost spent on that activity to obtain the utility. Both cost and utility values are positive real numbers ($co, ut \in R_+$). A ***triple*** $(\boldsymbol{a}, \boldsymbol{co}, \boldsymbol{ut})$ is called a quantitative-item (or briefly *q-item*). Let $E' \overset{def}{=} \{(a_i, co_i, ut_i) | a_i \in E, co_i \in R_+, ut_i \in R_+\}$ be called a *q-itemset* consisting of q-items, where $E = proj(E')$ is called the *projected* itemset of E'. A *q-sequence* $\alpha' = E'_1 \rightarrow E'_2 \rightarrow \ldots \rightarrow E'_p$ is a list of q-itemsets $E'_k, k = 1, \ldots, p$. The length and size of a *q-sequence* α' are respectively the total numbers of items and q-itemsets in α', and are respectively denoted as $length(\alpha')$ and $size(\alpha')$. If a q-sequence is *null* (denoted as <>), then its size is zero. A *quantitative sequence database* (QSDB) $\mathcal{D}'$ is a finite set of tuples <*SID*, Ψ'>, where *SID* is an identifier of input q-sequence Ψ'. Let $\alpha = proj(\alpha') \overset{def}{=} proj(E'_1) \rightarrow proj(E'_2) \rightarrow \ldots \rightarrow proj(E'_p)$ define a *projected* sequence of a q-sequence α' and $\mathcal{D} = proj(\mathcal{D}') \overset{def}{=} \{proj(\Psi'_i) \mid \Psi'_i \in \mathcal{D}'\}$ denote a *projected sequence database* (SDB) of $\mathcal{D}'$.

Definition 1 (*Utility and cost of q-elements*). The *utilities* of a q-item (a, c, u), q-itemset $E' = \{(a_{i_1}, co_{i_1}, ut_{i_1}), \ldots, (a_{i_m}, co_{i_m}, ut_{i_m})\}$, q-sequence $\alpha' = E'_1 \rightarrow E'_2 \rightarrow \ldots \rightarrow E'_p$ and $\mathcal{D}'$ are respectively defined and denoted as $u((a, co, ut)) \overset{def}{=} ut$, $u(E') \overset{def}{=} \sum_{j=1..m} u_{i_j}$, $u(\alpha') \overset{def}{=} \sum_{i=1..p} u(E'_i)$ and $u(\mathcal{D}') \overset{def}{=} \sum_{\Psi' \in \mathcal{D}'} c(\Psi')$. Similarly, the *costs* of (a, co, ut), E', α' and $\mathcal{D}'$ are defined and denoted as $c((a, co, ut)) \overset{def}{=} co$, $c(E') \overset{def}{=} \sum_{j=1..m} c_{i_j}$, $c(\alpha') \overset{def}{=} \sum_{i=p} c(E'_i)$ and $c(\mathcal{D}') \overset{def}{=} \sum_{\Psi' \in \mathcal{D}'} c(\Psi')$, respectively.

For example, a QSDB $\mathcal{D}'$ is depicted in Table 1, which will be used as a running example. The utility and cost of $\alpha' = (d, 3, 50) \rightarrow (a, 14, 4)(c, 1, 10)(f, 2, 36)$ are respectively $u(\alpha') = 50 + 4 + 10 + 36 = 100$ and $c(\alpha') = 3 + 14 + 1 + 2 = 20$.

Let $\alpha' = E'_1 \rightarrow E'_2 \rightarrow \ldots \rightarrow E'_p$, $\beta' = F'_1 \rightarrow F'_2 \rightarrow \ldots \rightarrow F'_q$ be any two q-sequences, and $\alpha = E_1 \rightarrow E_2 \rightarrow \ldots \rightarrow E_p$ and $\beta = F_1 \rightarrow F_2 \rightarrow \ldots \rightarrow F_q$ be projected sequences of α' and β', respectively.

Definition 2 (*Extensions of a sequence*). The *i-extension* (or *s-extension*) of α and β is defined and denoted as $\alpha \circ_i \beta \overset{def}{=} E_1 \rightarrow E_2 \rightarrow \ldots \rightarrow (E_p \cup F_1) \rightarrow F_2 \rightarrow \ldots \rightarrow F_q$, where $a \prec b, \forall a \in E_p, \forall b \in F_1$

Table 1 QSDB $\mathcal{D}'$

SIDDD	Input q-sequence
Ψ_1'	$(d,3,80)\rightarrow(a,10,5)(c,2,40)\rightarrow(a,7,3)(b,8,25)(f,1,3)\rightarrow(e,4,1)$
Ψ_2'	$(b,25,35)\rightarrow(c,7,2)(e,9,6)\rightarrow(d,5,1)\rightarrow(d,1,4)\rightarrow(g,35,45)\rightarrow(a,3,1)(f,2,9)$
Ψ_3'	$(d,4,8)\rightarrow(a,23,7)(c,3,35)(e,12,15)\rightarrow(d,7,22)\rightarrow(a,8,9)(f,3,72)$
Ψ_4'	$(c,21,5)(e,24,6)\rightarrow(a,13,3)\rightarrow(d,3,50)\rightarrow(a,4,5)(c,2,40)\rightarrow(a,14,4)(c,1,10)(f,2,36)$

(or $\alpha \diamond_s \beta \stackrel{def}{=} E_1 \rightarrow E_2 \rightarrow \ldots \rightarrow E_p \rightarrow F_1 \rightarrow F_2 \rightarrow \ldots \rightarrow F_q$, respectively). A *forward extension* (or briefly extension) of α with β, denoted as $\gamma = \alpha \diamond \beta$, can be either $\alpha \diamond_i \beta$ or $\alpha \diamond_s \beta$.

Definition 3 (*Partial order relations over q-sequences and sequences*). Consider any two q-itemsets $E' = \{(a_{i_1}, co_{i_1}, ut_{i_1}), \ldots, (a_{i_m}, co_{i_m}, ut_{i_m})\}$, $F' = \{(a_{j_1}, co_{j_1}, ut_{j_1}), \ldots, (a_{j_n}, co_{j_n}, ut_{j_n})\}$, and $m \le n$. Then, E' is said to be contained in F' (denoted as $E' \sqsubseteq F'$), if there exist natural numbers $1 \le k_1 < k_2 < \ldots < k_m \le n$ such that $a_{i_l} = a_{j_{k_l}}$, $co_{i_l} = co_{j_{k_l}}$ and $ut_{i_l} = ut_{j_{k_l}}$, $\forall l = 1, \ldots, m$. Similarly, α' is said to be *contained* in β' and denoted as $\alpha' \sqsubseteq \beta'$ (or β' is called a super q-sequence of α') if $p \le q$ and there exist p positive integers, $1 \le j_1 < j_2 < \ldots < j_p \le q$: $E'_k \sqsubseteq F'_{j_k}, \forall k = 1, \ldots, p$; and $\alpha' \sqsubset \beta' \Leftrightarrow (\alpha' \sqsubseteq \beta' \wedge \alpha' \ne \beta')$. Similarly, the following inclusion relation over all sequences is also defined: $\alpha \sqsubseteq \beta$ or $\beta \sqsupseteq \alpha$ (β is called a *super* sequence of α) if $\exists p$ positive integers, $1 \le j_1 < j_2 < \ldots < j_p \le q$: $E_k \subseteq F_{j_k}$, $\forall k = 1, \ldots, p$, and $\alpha \sqsubset \beta$ (β is called a *proper super* sequence of α) $\Leftrightarrow (\alpha \sqsubseteq \beta \wedge \alpha \ne \beta)$. The q-sequence β' *contains* the sequence α (or β' is a *super q-sequence* of α), denoted as $\alpha \sqsubseteq \beta'$ or $\beta' \sqsupseteq \alpha$, if $proj(\beta') \sqsupseteq \alpha$, and $\beta' \sqsupset \alpha$, if $proj(\beta') \sqsupset \alpha$. Let $\rho(\alpha) \stackrel{def}{=} \{\Psi' \in \mathcal{D}' \mid \Psi' \sqsupseteq \alpha\}$ denote the set of all input q-sequences containing α. The *support* of α is defined as the number of input super q-sequences of α, that is $supp(\alpha) = |\rho(\alpha)|$.

For instance, for $\beta = d \rightarrow ac \rightarrow af$ and $\Psi_1 = proj(\Psi_1') = d \rightarrow ac \rightarrow abf \rightarrow e$, then $\Psi_1 \sqsupset \beta$ and $\Psi_1' \sqsupset \beta$. Similarly, $\Psi_3' \sqsupset \beta$ and $\Psi_4' \sqsupset \beta$, so $\rho(\beta) = \{\Psi_1', \Psi_3', \Psi_4'\}$ and $supp(\beta) = 3$.

Note that because a sequence α may have multiple occurrences in each input q-sequence, the utility and cost values of α can be calculated in many ways. For instance, $\alpha = d \rightarrow ac$ appears twice in Ψ_4', because $(d, 3, 50) \rightarrow (a, 4, 5)(c, 2, 40) \sqsupseteq \alpha$ and $(d, 3, 50) \rightarrow (a, 14, 4)(c, 1, 10)$ contain α with two different utility values of 95 and 64, and two cost values of 9 and 18.

To formalize the proposed problem, for each $\Psi' \in \rho(\alpha)$, let $\mathcal{O}(\alpha, \Psi') \stackrel{def}{=} \{\alpha' \mid \alpha' \sqsubseteq \Psi' \wedge proj(\alpha') = \alpha\}$ be the set of all occurrences of α in Ψ'. For instance, $\mathcal{O}(\alpha, \Psi_4') = \{(d,50)\rightarrow(a,5)\ (c,40), (d,50)\rightarrow(a,4)\ (c,10)\}$. This paper defines the average utility and the average cost of a sequence in maximum form since most studies on high utility sequence mining have used that form, which was proposed in [28]. The definitions are as follows.

Definition 4 (*Average utility and average cost of a sequence*).

a. The (*maximum*) utility of a sequence α in Ψ_i' (or in $\mathcal{D}'$) [12] is denoted as $u(\alpha, \Psi_i')$ and defined as $u(\alpha, \Psi_i') \stackrel{def}{=} \max\{c(\alpha') \mid \alpha' \in \mathcal{O}(\alpha, \Psi_i')\}$ ($u(\alpha, \mathcal{D}')$ or briefly $u(\alpha) \stackrel{def}{=} \sum_{\Psi_i' \in \rho(\alpha)} u(\alpha, \Psi_i')$). Similarly, the (*maximum*) *cost* of α in Ψ_i' (or in $\mathcal{D}'$) is defined as $c(\alpha, \Psi_i') \stackrel{def}{=} \max\{c(\alpha') \mid \alpha' \in \mathcal{O}(\alpha, \Psi_i')\}$ or briefly $c(\alpha) \stackrel{def}{=} \sum_{\Psi_i' \in \rho(\alpha)} c(\alpha, \Psi_i')$).
b. The (*maximum*) *average utility* and *average cost* of α (among input q-sequences containing α) are denoted as $au(\alpha)$ and $ac(\alpha)$, and defined as $au(\alpha) \stackrel{def}{=} u(\alpha)/supp(\alpha)$ and $ac(\alpha) \stackrel{def}{=} c(\alpha)/supp(\alpha)$, respectively.

Problem definition Given user-predefined *minimum utility* and *minimum support* thresholds (*minu*, *mins*), and a *maximum cost* threshold (*maxc*), a sequence α is said to be a *frequent*, *low* (*average*) *cost* and *high* (*average*) *utility* (or briefly FLCHU) sequence if $supp(\alpha) \ge mins$, $ac(\alpha) \le maxc$ and $au(\alpha) \ge minu$, respectively. The problem of FLCHU sequence mining (*FLCHUSM*) in a QSDB $\mathcal{D}'$ is to mine the set $FLCHUS \stackrel{def}{=} \{\alpha \mid \text{supp}(\alpha) \ge mins \wedge au(\alpha) \ge minu \wedge ac(\alpha) \le maxc\}$.

The thresholds *mins*, *minu* and *maxc* can also be given in another equivalent percentage form: $mins\ (\%) \stackrel{def}{=} \frac{mins}{|\mathcal{D}'|} \times 100$, $minu\ (\%) \stackrel{def}{=} \frac{minu}{maxUSeq} \times 100$ and $maxc\ (\%) \stackrel{def}{=} \frac{maxc}{maxCSeq} \times 100$, where $maxUSeq \stackrel{def}{=} max\{u(\Psi') \mid \Psi' \in \mathcal{D}'\}$ and $maxCSeq \stackrel{def}{=} max\{c(\Psi') \mid \Psi' \in \mathcal{D}'\}$. Obviously, a sequence α is a FLCHU sequence if and only if $supp(\alpha) \ge \frac{mins(\%)}{100} \times |\mathcal{D}'|$, $ac(\alpha) \le \frac{maxc(\%)}{100} \times maxCSeq$ and $au(\alpha) \ge \frac{minu(\%)}{100} \times maxUSeq$.

In particular, when *maxc* is sufficiently large (e.g. $\max\{u(\Psi_i') \mid \Psi_i' \in \mathcal{D}'\}$), if $mins = 1$ (or $minu = 0$) we obtain the more simple problem of *high average utility* (or the traditional problem of *frequent*, respectively) sequence mining.

For example, for $minu = 40$, $maxc = 6.6$, $mins = 3$ and $\alpha = d \rightarrow c$, we have $\rho(\alpha) = \{\Psi_1', \Psi_3', \Psi_4'\}$ and $\mathcal{O}(\alpha, \Psi_4') = \{(d,3,50)\rightarrow(c,2,40), (d,3,50)\rightarrow(c,1,10)\}$, so $supp(\alpha) = 3 \ge mins$ and $u(\alpha, \Psi_4') = max\{90, 60\} = 90$. Similarly, $u(\alpha, \Psi_1') = 120$ and $u(\alpha, \Psi_3') = 43$. Hence, $u(\alpha) = 253$ and $au(\alpha) = 253/3 \approx 84.33 \ge minu$. Furthermore, $c(\alpha, \Psi_4') = max\{5, 4\} = 5$ and similarly,

$c(\alpha, \Psi_1') = 5$, $c(\alpha, \Psi_3') = 7$, thus, $ac(\alpha) = 17/3 \approx 5.67 \leq maxc$, i.e. α is a FLCHU sequence. For another sequence $\beta = d \rightarrow f$, we also have that $au(\beta) = 69 \geq minu$, $ac(\beta) = 6.5 \leq maxc$ and $supp(\beta) = 4 \geq mins$. Furthermore, we have the result set $FLCHUS = \{d \rightarrow c, d \rightarrow f\}$.

3.1 Discussion

a. An interesting question is how to set the three *minu*, *mins* and *maxc* thresholds of the proposed problem to find interesting patterns for a given application. As for other pattern mining tasks, the proposed problem is an unsupervised data mining task that is aimed at exploring the data to find patterns that can reveal interesting insights. To find patterns, a user must specify constraints on these patterns using the three parameters, and how to set these parameters depends on the user's needs and the data. Different users may be interested in finding patterns meeting different requirements, and a user may want to run an algorithm more than once to explore the patterns of a database. The maximum cost threshold *maxc* has a clear meaning to the user as it represents the minimum amount of effort or resource that the user is willing to spend. For example, a learner may not be interested in spending more than 1 h on average for each learning activity and may thus set *maxc* = 1. Similarly, a tourist may not be interested in spending more than 10 $ on average per tourist attraction and may set *maxc* = 10. The minimum utility threshold *minu* also has a clear meaning for the users. It represents the amount of benefits that the user wishes to obtain. For instance, a student may not want to have an average score for learning activities below 60% to not risk failing a course, and a more motivated student may not want to have a score below 80%. In such case, *minu* can be set to 60% and 80%, respectively. Another example is a tourist that may set *minu* = 4 to not visit tourist attractions that have less than 4 stars in terms of user satisfaction on average. The third parameter, which is *mins*, is only used to filter out patterns that may be considered as noise because they do not appear often enough. How to set *mins* depends on the dataset and may thus be set empirically. If it is too high, no patterns may be found and if it is too low, some irrelevant patterns may be discovered.

Another remark is that the higher the *mins*, *minu* thresholds are set and the lower *maxc* value is, the smaller the number of FLCHU patterns is found, and those patterns may usually be more interesting. However, for too small *maxc* or too large *minu* and *mins* values, the set of mined patterns may be empty. Thus, a trade-off among *maxc*, *minu* and *mins* is necessary and this can require running the algorithm a few times to find interesting patterns. Thus, setting the parameters depends on dataset of each concrete application and on the users' needs.

b. In [12, 40], two different measures of the average cost of a sequence α were considered, defined as $ac_{first}(\alpha) \stackrel{def}{=} \sum_{\Psi_i' \in \rho(\alpha)} c_{first}(\alpha, \Psi_i') / supp(\alpha)$ and $ac_{last}(\alpha) \stackrel{def}{=} \sum_{\Psi_i' \in \rho(\alpha)} c_{last}(\alpha, \Psi_i') / supp(\alpha)$, which are simply based on the sub-q-sequences α_{first}' and α_{last}' that are the first and last occurrences of α in Ψ_i', respectively, where $c_{first}(\alpha, \Psi_i') = c(\alpha_{first}')$ and $c_{last}(\alpha, \Psi_i') = c(\alpha_{last}')$. It is found that $\{ac_{first}(\alpha), ac_{last}(\alpha)\} \leq ac(\alpha)$. Besides, if each item only appears at most once in each input quantitative sequence, $ac(\alpha)$ presented in Definition 4 becomes equal to $ac_{first}(\alpha)$ and $ac_{last}(\alpha)$.

Moreover, the average utility presented in the two above papers is defined differently as $au_s(\alpha) \stackrel{def}{=} \sum_{\Psi_i' \in \rho(\alpha)} u(\Psi_i') / supp(\alpha)$, which is often much larger than $au(\alpha)$ in Definition 4, $au_s(\alpha) \geq au(\alpha)$. Thus, the pairs of functions $(ac_{first}(\alpha), au_s(\alpha))$, $(ac_{last}(\alpha), au_s(\alpha))$ and the corresponding problems of [12, 40] are totally different from the average cost and average utility functions $(ac(\alpha), au(\alpha))$ and the *FLCHUSM* problem proposed in the current paper.

It is also found that the set *FLCHUS* of FLCHU sequences defined in this paper is much smaller compared to the two previous papers [12, 40]. *Indeed*, note that the definition $au_s(\alpha) \stackrel{def}{=} \sum_{\Psi_i' \in \rho(\alpha)} u(\Psi_i') / supp(\alpha)$ is based on the overestimated utility $u(\Psi_i')$ of the exact utility $u(\alpha, \Psi_i')$ proposed in this manuscript, $u(\Psi_i') \geq u(\alpha, \Psi_i')$. Therefore, if α has a high utility w.r.t. *au*, then all sequences β in the same equivalence class $[\alpha]$ also have high utilities w.r.t. au_s, where $[\alpha] \stackrel{def}{=} \{\beta \mid \rho(\beta) = \rho(\alpha)\}$. The reason is that since $supp(\beta) = supp(\alpha)$, it follows that $au_s(\beta) = \sum_{\Psi_i' \in \rho(\beta)} u(\Psi_i') / supp(\beta) = \sum_{\Psi_i' \in \rho(\alpha)} u(\Psi_i') / supp(\alpha) = au_s(\alpha) \geq \sum_{\Psi_i' \in \rho(\alpha)} u(\alpha, \Psi_i') / supp(\alpha) = au(\alpha) \geq minu$. Moreover, if $ac(\alpha) \leq maxc$ and $au(\alpha) \geq minu$, then $max\{ac_{first}(\alpha), ac_{last}(\alpha)\} \leq maxc$ and $au_s(\alpha) \geq minu$. As a consequence, the cardinality of the set *FLCHUS* according to the pair $(ac(\alpha), au(\alpha))$ is often much smaller compared to those according to the two pairs $(ac_{first}(\alpha), au_s(\alpha))$ and $(ac_{last}(\alpha), au_s(\alpha))$.

For the running example, for *mins* = 50%, *minu* = 62% and *maxc* = 98%, then the cardinality $|FLCHUS|$ according to the pairs (ac, au), (ac_{first}, au_s) and (ac_{last}, au_s) are respectively 4, 60 and 60. Consider the lowest threshold *mins* = 25% and another *minu* = 92%. Then, for *maxc* = 37%, the cardinality $|FLCHUS|$ according to the pairs (ac, au), (ac_{first}, au_s) and (ac_{last}, au_s) are respectively 1, 211 and 207. In addition, for the higher *maxc* = 67%, then the cardinality $|FLCHUS|$ according to the pairs (ac, au), (ac_{first}, au_s) and (ac_{last}, au_s) are respectively 5, 454 and 452, i.e. the difference of their values is much greater.

A similar observation can be made on larger real-life QSDBs such as Snake shown in Fig. 8. In other words, using (ac_{first},au_s) or (ac_{last},au_s) presented in our two previous papers can result in difficulty for users in analyzing the final set, especially for low *mins* and high *maxc* values.

To solve a pattern mining problem efficiently, it is necessary to find properties to reduce the search space (to avoid considering all sequences). The support *supp* is *anti-monotonic*, which is a nice property for reducing the search space, widely used in frequent sequence mining. Generally, an arbitrary function of sequences *mes* is called anti-monotonic (or monotonic) if for any two sequences α and β such that $\alpha \sqsubseteq \beta$, then $mes(\beta) \leq mes(\alpha)$ (or $mes(\beta) \geq mes(\alpha)$, respectively). The *average* utility and cost functions, however, are neither *anti-monotonic* nor *monotonic*. This is illustrated by considering four sequences $\alpha = ce \rightarrow f$, $\beta = ce \rightarrow af$, $\gamma = d \rightarrow ce \rightarrow f$ and $\delta = b \rightarrow ce \rightarrow f$. It is found that $au(\beta) = 66.67 > au(\alpha) = 62 > au(\delta) = 52$ but $\delta \sqsupset \alpha \sqsubset \beta$, and that $ac(\beta) = 36 > ac(\alpha) = 27.67 > ac(\gamma) = 22$ while $\gamma \sqsupset \alpha \sqsubset \beta$. The proposed problem can be much more challenging than frequent sequence mining since additional time is required for the calculation of the cost and utility of candidate sequences. Thus, it is important to design upper bounds on *au* and lower bounds on *ac* that satisfy respectively *anti-monotonic* and *monotonic* properties for pruning unpromising candidates early during the process of mining *FLCHUS*.

4 Designing UBs on *au* and an LB on *ac*

This section presents novel upper bounds on *au* and lower bounds on *ac* with the goal of efficiently reducing the search space. The concept of *ending* and *remaining sequence* of a sub-sequence in a *q*-sequence is firstly introduced. Then, the upper bounds are presented.

Assume that $\alpha = E_1 \rightarrow E_2 \rightarrow \ldots \rightarrow E_p \sqsubseteq \Psi' = F'_1 \rightarrow F'_2 \rightarrow \ldots \rightarrow F'_q$, i.e. there exist a *q*-sequence $\alpha' = E'_1 \rightarrow E'_2 \rightarrow \ldots \rightarrow E'_p \in \mathcal{O}(\alpha, \Psi')$ and p positive integers: $1 \leq i_1 < i_2 < \ldots < i_p \leq q$ such that $E'_k \subseteq F'_{i_k}, \forall k = 1, \ldots, p$. Then, the index i_p is said to be an *ending* of α or α' in Ψ', denoted as $end(\alpha, \Psi')$ or $end(\alpha', \Psi')$, and the last item of α in F'_{i_p} is called an *ending item* and denoted as e_{i_p}. Then, the *remaining* *q*-sequence of α in Ψ' w.r.t. the ending i_p is the rest of Ψ' after α' (or after the ending item e_{i_p}) and denoted as $rem(\alpha, \Psi', \alpha')$ or $rem(\alpha, \Psi', i_p)$. If $\alpha = <>$, then as a convention, $rem\left(<>, \Psi', i_p^*\right) = \Psi'$. For any $\Psi' \in \rho(\alpha)$ and $\alpha' \in \mathcal{O}(\alpha, \Psi')$, let α'_{fist} be the (*unique*) *q*-sequence α' in $\mathcal{O}(\alpha, \Psi')$ that is according to the first occurrence of α in Ψ'. Note that α'_{fist} only depends on α and Ψ' but not on α'.

For example, consider the sequence $\alpha = d \rightarrow c$ of which two endings in Ψ'_4 are 4 and 5. Thus, its first ending $i_p^* = FEnd(\alpha, \Psi'_4)$ is 4, $\alpha'_{fist} = (d,3,50) \rightarrow (c,2,40)$, $rem\left(\alpha, \Psi'_4, i_p^*\right) = (a,14,4)(c,1,10)(f,2,36)$ and $rem(\alpha, \Psi'_4, 5) = (f,2,36)$.

4.1 Designing two upper bounds on *au*

The next paragraph defines the concept of upper bound, explains how upper bounds can be compared, and proposes two novel upper bounds on the average utility of a sequence. These upper bounds help to significantly reduce the search space.

A function *ub* is called an *upper bound* (UB) on a function *mes* (of sequences) if $mes(\alpha) \leq ub(\alpha)$, for any sequence α. For any two functions ub_1 and ub_2, ub_1 is said to be *tighter* than ub_2 if $ub_1(\alpha) \leq ub_2(\alpha)$, $\forall \alpha$. Besides, ub_1 is said to be *strictly tighter* than ub_2 if $ub_1(\alpha) \leq ub_2(\alpha)$, $\forall \alpha$, and there exists a frequent sequence β: $ub_1(\beta) < ub_2(\beta)$. Lastly, ub_1 and ub_2 are *incomparable* if there exist two different frequent sequences $\alpha \neq \beta$ such that $ub_1(\alpha) < ub_2(\alpha)$ and $ub_1(\beta) > ub_2(\beta)$.

Definition 5 (*Upper bounds on au*). For any *frequent* sequence α and $\Psi'_i \in \rho(\alpha)$, we define gradually tighter UBs on $u(\alpha, \Psi'_i)$ as $u(\Psi'_i)$ and $ub(\alpha, \Psi'_i) \stackrel{def}{=} max\{ub_{rem}(\alpha'), \alpha' \in \mathcal{O}(\alpha, \Psi'_i)\}$, where $ub_{rem}(\alpha') \stackrel{def}{=} u(\alpha') + u(rem(\alpha, \Psi'_i, \alpha'))$ and the vector of *input q-sequence utilities* (*ISU*) is defined as $ISU \stackrel{def}{=} (u(\Psi'_i), \Psi'_i \in \mathcal{D}')$. Assume that after sorting in *descending* order the values of the series $(u_i \stackrel{def}{=} ub(\alpha, \Psi'_i), \Psi'_i \in \rho(\alpha))$ and $(u'_i \stackrel{def}{=} u(\Psi'_i), \Psi'_i \in \rho(\alpha))$, we obtain $(u_{i_j}, j = 1, \ldots, s)$ and $(u''_{i_j}, j = 1..s)$ with $s = supp(\alpha) = |\rho(\alpha)| \geq m \stackrel{def}{=} mins$, and define $S_k \stackrel{def}{=} \sum_{1 \leq j \leq k} u_{i_j}$ and $S'_k \stackrel{def}{=} \sum_{1 \leq j \leq k} u''_{i_j}$ as the sum of *k largest* values of corresponding sequences, for $k = 1, \ldots, s$. Then, two tighter UBs on *au* are defined and denoted as $AMUB(\alpha) \stackrel{def}{=} \frac{1}{m} S'_m$ and $FUB(\alpha) \stackrel{def}{=} \frac{1}{m} S_m$.

For example, for the sequence $\alpha = d$ and $m = mins = 3$, then $\rho(\alpha) = \mathcal{D}'$, $supp(\alpha) = 4 \geq m$ and $au(\alpha) = 39$. For $\Psi'_2 \in \rho(\alpha)$, $\mathcal{O}(\alpha, \Psi'_2) = \{\alpha'_1 = (d,5,1), \alpha'_2 = (d,1,4)\}$, $rem(\alpha, \Psi'_2, \alpha'_1) = (d,1,4) \rightarrow (g,35,45) \rightarrow (a,3,1)\ (f,2,9)$ and $rem(\alpha, \Psi'_2, \alpha'_2) = (g,35,45) \rightarrow (a,3,1)\ (f,2,9)$. Hence, $ub(\alpha, \Psi'_2) = max\{ub_{rem}(\alpha'_1), ub_{rem}(\alpha'_2)\} = max\{60, 59\} = 60$, and similarly, $ub(\alpha, \Psi'_1) = 157$, $ub(\alpha, \Psi'_3) = 168$, $ub(\alpha, \Psi'_4) = 145$. Thus, $FUB(\alpha) = \frac{1}{3} S_3 = (168 + 157 + 145)/3 \approx 156.67$. Moreover, we have the vector $ISU = (157, 103, 168, 159)$, so $AMUB(\alpha) = \frac{1}{3} S''_3 = (168 + 159 + 157)/3 \approx 161.33$. Thus, $au(\alpha) \leq FUB(\alpha) \leq AMUB(\alpha)$.

The following theorem presents important anti-monotone-like properties of the *AMUB* and *FUB* UBs. These nice properties will be used in efficient pruning strategies presented in Section 4.3 to reduce the search space. It will be shown that

although *AMUB* is a looser UB on *au* than *FUB*, using both to mine patterns is desirable.

Theorem 1 (*Relation between the AMUB and FUB UBs, and their anti-monotone-like* ($\mathcal{AML}$) *properties*)

a. *AMUB* and *FUB* UBs on *au* are *gradually strictly tighter*, i.e. $FUB(\alpha) \leq AMUB(\alpha)$, $\forall \alpha$ and there exists a sequence β: $FUB(\beta) < AMUB(\beta)$.
b. (*i*). $\mathcal{AM}(AMUB)$, or *AMUB* is *anti-monotonic,* i.e. $AMUB(\beta) \leq AMUB(\alpha)$, for any *super sequence* β of α.

(*ii*). $\mathcal{AMF}(FUB)$, or *FUB* is anti-monotonic w.r.t. *forward extension,* i.e. $FUB(\beta) \leq FUB(\alpha)$, for any *frequent forward* extension β of α. However, *not*$(\mathcal{AM}(FUB))$.

It is clear that, for any UB *ub* on *au*, we have ($\mathcal{AM}(ub) \Rightarrow \mathcal{AMF}(ub)$), i.e. the $\mathcal{AMF}$ property is weaker than $\mathcal{AM}$. Then, the properties $\mathcal{AM}$ and $\mathcal{AMF}$ are called anti-monotone-like ($\mathcal{AML}$).

Proof.

a. Since for any $\Psi_i' \in \rho(\alpha)$ and $\alpha' \in \mathcal{O}(\alpha, \Psi_i')$, $u(\alpha') \leq u(\alpha') + u(rem(\alpha, \Psi_i', \alpha')) = ub_{rem}(\alpha') \leq u(\Psi_i')$, then $u(\alpha, \Psi_i') \leq ub(\alpha, \Psi_i') \leq u(\Psi_i')$. Due to $m \times u_{i_j} \leq S_m, \forall j = m+1, \ldots, s$, with $s = |\rho(\alpha)| \geq m = mins$, so $au(\alpha) \stackrel{def}{=} \frac{1}{s} \sum_{\Psi_i' \in \rho(\alpha)} u(\alpha, \Psi_i') \leq \frac{1}{s} S_s = \frac{1}{m \times s} \left(m \times S_m + \sum_{m+1 \leq j \leq s} (m \times u_{i_j})\right) \leq \frac{1}{m \times s} \left(m \times S_m + \sum_{m+1 \leq j \leq s} S_m\right) = \frac{1}{m} S_m = FUB(\alpha) \leq \frac{1}{m} S'_m = AMUB(\alpha)$.

b. (*i*). For any *frequent* super sequence β of α, since $u(\Psi_i') | \Psi_i' \in \rho(\beta) \} \subseteq u(\Psi_i') | \Psi_i' \in \rho(\alpha)$, $AMUB(\alpha) = \frac{1}{m} S''_m \geq AMUB(\beta)$, i.e. $\mathcal{AM}(AMUB)$.

(*ii*). For any *frequent forward* extension $\beta = \alpha \diamond \delta \sqsupseteq \alpha$, $\Psi_i' \in \rho(\beta) \subseteq \rho(\alpha)$ and $\beta' \in \mathcal{O}(\beta, \Psi_i')$, $\exists \alpha' \in \mathcal{O}(\alpha, \Psi_i')$: $\alpha' \sqsubseteq \beta' \sqsubseteq \Psi_i'$. On the other hand, since $ub_{rem}(\beta') \leq ub_{rem}(\alpha')$, $ub(\beta, \Psi_i') \leq ub(\alpha, \Psi_i')$ and $FUB(\beta) = \frac{1}{m} \sum_m$ *largest values of* $\{b(\beta, \Psi_i') | \Psi_i' \in \rho(\beta)\}$ $ub(\beta, \Psi_i') \leq \frac{1}{m} \sum_m$ *largest values of* $\{ub(\alpha, \Psi_i') | \Psi_i' \in \rho(\alpha)\}$ $ub(\alpha, \Psi_i') = FUB(\alpha)$, i.e. $\mathcal{AMF}(FUB)$.

For example, for $m = mins = 3$, a *forward* extension $\beta = d \rightarrow c$ and a *super* sequence $\gamma = c \rightarrow d$ of the above sequence $\alpha = d$, then γ is not a forward extension of α. We also have $FUB(\beta) = 151$, $FUB(\gamma) = 151$, $AMUB(\beta) = AMUB(\alpha) = 161.33$ and $AMUB(\gamma) = 143.33$. Thus, $FUB(\beta) \leq FUB(\alpha) = 156.67$, $AMUB(\beta) \leq AMUB(\alpha)$ and $AMUB(\gamma) \leq AMUB(\alpha)$, but $FUB(\gamma) > FUB(\alpha)$, i.e. *not*$(\mathcal{AM}(FUB))$.

4.2 Designing a lower bound on *ac*

The previous section has defined upper bounds on the average utility. To be able to further reduce the search space using the average cost, this section proposes a lower bound on the average cost *ac* that satisfies a property called *monotonic* w.r.t. *forward extension*, which is said to be *monotone-like* and it is weaker than the well-known monotonic property.

Definition 6 (*Lower bound on ac*). For any *frequent* sequence α and $\Psi_i' \in \rho(\alpha)$, let LB $lb(\alpha, \Psi_i') \stackrel{def}{=} c(\alpha'_{first})$ be a lower bound on $c(\alpha, \psi_i')$, i.e. $c(\alpha'_{first}) \leq c(\alpha, \Psi_i')$. Assume that after sorting in *ascending* order the values of the series $(c_i \stackrel{def}{=} lb(\alpha, \Psi_i'), \Psi_i' \in \rho(\alpha))$, we obtain $(c_{i_j}, j = 1, \ldots, s)$ with $s = |\rho(\alpha)| \geq m \stackrel{def}{=} mins$ and define $T_k \stackrel{def}{=} \sum_{1 \leq j \leq k} c_{i_j}$ as the sum of the k *smallest* values of $(c_i, \Psi_i' \in \rho(\alpha))$, for $k = 1, \ldots, s$. Then, a lower bound (LB) on *ac* is defined and denoted as $Flb(\alpha) \stackrel{def}{=} \frac{1}{m} T_m$.

Theorem 2 (*Monotone-like* ($\mathcal{ML}$) *property of the Flb LB*)

a. *Flb* is an LB on *ac*, i.e. $Flb(\alpha) \leq ac(\alpha)$.
b. $\mathcal{MF}(Flb)$, or *Flb* is *monotonic* w.r.t. *forward extension*, i.e. $Flb(\beta) \geq Flb(\alpha)$, for any *frequent forward* extension β of α. Then, the property $\mathcal{MF}$ is called *monotone-like* ($\mathcal{ML}$).

Proof. For the sake of brevity, let $m \stackrel{def}{=} mins$.

a. For any $\Psi_i' \in \rho(\alpha)$, by the definition of $c(\alpha, \Psi_i')$, it is clear that $c(\alpha, \Psi_i') \geq c(\alpha'_{first}) = lb(\alpha, \Psi_i')$. Due to $m \times c_{i_j} \geq T_m$, $\forall j = m+1, \ldots, s$, with $s = |\rho(\alpha)| \geq m$, we have $ac(\alpha) \stackrel{def}{=} \frac{1}{s} \sum_{\Psi_i' \in \rho(\alpha)} c(\alpha, \Psi_i') \geq \frac{1}{s} T_s = \frac{1}{m \times s} \left(m \times T_m + \sum_{m+1 \leq j \leq s} (m \times c_{i_j})\right) \geq \frac{1}{m \times s} \left(m \times T_m + \sum_{m+1 \leq j \leq s} T_m\right) = \frac{1}{m} T_m = Flb(\alpha)$, i.e. *Flb* is an LB on *ac*.
b. For any *frequent forward* extension $\beta = \alpha \diamond \delta \sqsupseteq \alpha$, $\Psi_i' \in \rho(\beta) \subseteq \rho(\alpha)$ and $\beta'_{first} \in \mathcal{O}(\beta, \Psi_i')$, $\exists \alpha'_{first} \in \mathcal{O}(\alpha, \Psi_i')$: $\alpha'_{first} \sqsubseteq \beta'_{first} \sqsubseteq \Psi_i'$. Moreover, since $lb(\alpha, \Psi_i') = c(\alpha'_{first}) \leq c(\beta'_{first}) = lb(\beta, \Psi_i')$, so $Flb(\beta) = \frac{1}{m} \sum_{m \text{ smallest values of }} \{lb(\beta, \Psi_i') | \Psi_i' \in \rho(\beta)\} lb(\beta, \Psi_i')$

$\geq \frac{1}{m}\sum_{m \text{ smallest values of } \{lb(\alpha,\Psi_i') \mid \Psi_i' \in \rho(\alpha)\}} lb(\alpha,\Psi_i')$) = $FLB(\alpha)$, i.e. $\mathcal{MF}(Flb)$.

For example, for $mins = 3$ and the sequence $\alpha = d$, then $\rho(\alpha) = \mathcal{D}'$, $supp(\alpha) = 4 \geq mins$ and $ac(\alpha) = 4.5$. For $\Psi_2' \in \rho(\alpha)$, $\alpha'_{first} = (d, 5, 1)$, $lb(\alpha, \Psi_2') = 5$, and similarly, $lb(\alpha, \Psi_1') = lb(\alpha, \Psi_4') = 3$, $lb(\alpha, \Psi_3') = 4$. Thus, $Flb(\alpha) = \frac{1}{3}T_3 = (3+3+4)/3 \approx 3.33$ and $ac(\alpha) \geq Flb(\alpha)$. For a *forward* extension $\beta = d \rightarrow c$ of $\alpha = d$, we also obtain $Flb(\beta) = 5.67$ and $Flb(\beta) \geq Flb(\alpha)$.

4.3 Designing pruning strategies

Based on the $\mathcal{AML}$ properties of the support, *supp*, and two *AMUB*, *FUB* UBs on *au*, the $\mathcal{ML}$ property of the *Flb* LB on *ac*, this subsection proposes *depth*, *width* pruning and *tightening* strategies to reduce the search space. Those will be presented in Theorem 3, below. For the sake of brevity, additional definitions are first introduced:

$$DepthPC(\alpha) \stackrel{def}{=} (FUB(\alpha) < minu \vee Flb(\alpha) > maxc) \text{ and}$$
$$WidthPC(\alpha) \stackrel{def}{=} (AMUB(\alpha) < minu \vee supp(\alpha) < mins)$$

as *depth* and *width* pruning conditions.

Furthermore, we define

$$I(\alpha) \stackrel{def}{=} \{y \in \mathcal{A} \mid y \succ lastItem(\alpha) \wedge AMUB(\alpha \circ_i y) \geq minu \wedge \text{supp}(\alpha \circ_i y) \geq mins\}$$

(and)

$$S(\alpha) \stackrel{def}{=} \{y \in \mathcal{A} \mid AMUB(\alpha \circ_s y) \geq minu \wedge \text{supp}(\alpha \circ_s y) \geq mins\}$$

as two sets of candidate items for *i*- and *s*-extensions of α. Then, $I(<>) \equiv S(<>) = \{y \in \mathcal{A} \mid AMUB(y) \geq minu \wedge \text{supp}(y) \geq mins\}$.

For example, for $minu = 40$, $maxc = 6.6$ and $mins = 3$, since $supp(g) = 1 < supp(b) = 2 < mins$, then b and $g \notin I(<>) = S(<>)$. Moreover, since for any $y \in \{a, c, d, e, f\}$, $AMUB(y) \geq minu$ and $supp(y) \geq mins$ (e.g. $AMUB(d) = 153 \geq minu$ and $supp(d) = 4 \geq mins$), then $S(<>) = \{a, c, d, e, f\}$.

The search space of *FLCHUSM* problem can be considered as a prefix-tree, where each node represents a sequence. The root of the prefix-tree is initialized as the null sequence, and children of a node α contain the *i*- or *s*-extensions of α. A branch in the prefix-tree starting from α, denoted as $branch(\alpha)$, consists of α and its extensions (its children).

Theorem 3 (*Depth, width pruning and tightening strategies*)

a. The ***Depth pruning strategy*** (DPS) relies on the *FUB* UB and *FLB* LB. For any sequence $\alpha \neq <>$, if $DepthPC(\alpha)$ then $au(\beta) < minu$ or $ac(\beta) > maxc$, for all (*forward*) extensions β of α, i.e. the $branch(\alpha)$ can be *deeply* pruned.

b. The ***Width pruning strategy*** (WPS) relies on the *AMUB* UB and *supp* measure. For any α (including <>), if $WidthPC(\alpha)$ then the $branch(\alpha)$ can be *deeply* pruned. In addition, the following ***tightening strategy*** (TS) can be also applied: $I(\alpha \circ_i x) \subseteq I(\alpha)$ and $I(\alpha \circ_s x) \cup S(\alpha \circ_i x) \cup S(\alpha \circ_s x) \subseteq S(\alpha)$, i.e. we can gradually tighten the two candidate sets *I* and *S* during the mining process.

Proof.

a. If $FUB(\alpha) < minu$ or $Flb(\alpha) > maxc$, then according to Theorem 1.b.(*ii*) and Theorem 2.b, $\forall \beta = \alpha \circ \delta \sqsupseteq \alpha \neq <>$, $\rho(\beta) \subseteq \rho(\alpha)$, $au(\beta) \leq FUB(\beta) \leq FUB(\alpha) < minu$ or $ac(\beta) \geq Flb(\beta) \geq Flb(\alpha) > maxc$, i.e. the whole $branch(\alpha)$ can be deeply pruned.

b. If $AMUB(\alpha) < minu$ or $supp(\alpha) < mins$, then by Theorem 1.b.(*i*) and $\mathcal{AM}(supp)$, $\forall \beta \sqsupseteq \alpha$ (including <>), $\rho(\beta) \subseteq \rho(\alpha)$, $au(\beta) \leq AMUB(\beta) \leq AMUB(\alpha) < minu$ or $supp(\beta) \leq supp(\alpha) < mins$, i.e. the $branch(\alpha)$ can be deeply pruned. In addition, due to $\mathcal{AM}(AMUB)$ and $\mathcal{AM}(supp)$, for instance, for any $y \in I(\alpha \circ_i x)$, then $y \succ x$, $AMUB(\alpha \circ_i x \circ_i y) \geq minu$ and $supp(\alpha \circ_i x \circ_i y) \geq mins$. Thus, $AMUB(\alpha \circ_i y) \geq AMUB(\alpha \circ_i x \circ_i y) \geq minu$ and $supp(\alpha \circ_i y) \geq supp(\alpha \circ_i x \circ_i y) \geq mins$, so $y \in I(\alpha)$, i.e. $I(\alpha \circ_i x) \subseteq I(\alpha)$. Similarly, the remaining assertions are also proven.

For instance, for $minu = 40$, $maxc = 6.6$, $mins = 3$, since $Flb(a) = 8.66$, $Flb(e) = 8.33$ are greater than *maxc* and $FUB(f) = 39 < minu$, branches $branch(a)$, $branch(e)$ and $branch(f)$ can be *deeply* pruned early. If we increase *minu* to 117, we have $I(<>) = S(<>) = \{a, c, d, e, f\}$. Because $I(c) \cup S(c) \subseteq I(<>)$, we have $AMUB(ce) = 116.66 < minu$ and $supp(cd) = 0 < supp(cf) = 1 < mins \leq supp(ce) = 3$, i.e. *ce* is *widely* pruned by using *AMUB* but not by *supp*, so $I(c) = \emptyset$. Similarly, since $AMUB(c \rightarrow d) = 116.66 < minu$ and $supp(c \rightarrow c) = supp(c \rightarrow e) = 1 < mins \leq supp(c \rightarrow d) = 3$, then $S(c) = \{a, f\}$. Thus, not only branches $branch(cd)$, $branch(ce)$, $branch(cf)$, $branch(c \rightarrow c)$, $branch(c \rightarrow d)$ and $branch(c \rightarrow e)$ are *deeply* pruned, but we can also apply the $\mathcal{TS}$ tightening strategy: $I(c \circ_i x) \subseteq I(c) = \emptyset$, $I(c \circ_s x) \cup S(c \circ x) \subseteq S(c) = \{a, f\}$. Hence, $I(c \circ_i x) = \emptyset$, i.e. it is not necessary to consider all *i*-extensions of *c* with any $x \in \{d, e, f\}$. In the same way, for example, for finding candidate items *y* in the set of *i*-extensions of $c \circ_s x$ with *y*, $I(c \circ_s x) = \{y \in \mathcal{A} \mid y \succ x \wedge AMUB(c \circ_s x \circ_i y) \geq minu \wedge \text{supp}(c \circ_s x \circ_i y) \geq mins\}$, we need to consider only one pair of items $(x, y) = (a, f)$. In other words, the *tightening* $\mathcal{TS}$ helps to reduce the search space and to speed-up the process of mining *FLCHUS*.

4.3.1 Optimization for mining *FLCHUS*

Besides the two efficient pruning $\mathcal{DPS}$ and $\mathcal{WPS}$ strategies, to further reduce the search space, we can additionally apply the two following strategies for reducing a QSDB and

tightening the two sets of candidate items for extensions considered by the mining process.

a. The ***Reducing QSDB strategy*** (*RedS*). For any item y of $\mathcal{A}$ that satisfies the width pruning condition *WidthPC*(y), we can reduce the original QSDB $\mathcal{D}'$ by removing y from $\mathcal{D}'$. Besides, we can also update and reduce the values of all upper bounds of remaining items. *Indeed*, since $\mathcal{AM}(AMUB)$ and $\mathcal{AM}(supp)$, for any sequence α containing y, $\alpha = \varepsilon \diamond y \diamond \delta$, we have $au(\alpha) \le AMUB(\alpha) \le AMUB(y) < minu$ or $supp(\alpha) \le supp(y) < mins$, i.e. α cannot be an FLCHU sequence. For instance, for $minu = 40$, $mins = 3$, we have the vector $ISU \triangleq \left(u\left(\Psi_i', i = 1,2,3,4\right)\right) = (157; 103; 168; 159)$. Since $supp(g) = 1 < supp(b) = 2 < mins$, we can remove two irrelevant items, b and g, from $\mathcal{D}$'. Then, in such reduced QSDB, we update UBs of remaining items, which can be tightened. First, we have the updated vector $ISU = (132; 23; 168; 159)$. Then, for the item a, for example, before removing b and g, values of the (*FUB*, *AMUB*) UBs are respectively (128.3, 161.3), and after removing these items, the updated values are (120, 153), i.e. the updated UB values can be reduced.

b. The ***Tightening* strategy** *CMAPS* based on *AMUB* and *supp*. To speed up the mining process, in the following we define *iCMAP* and *sCMAP*, inspired by the *CMAP* technique of FPM [2]. For each item x of $\mathcal{A}$, then $iCMAP(x) \triangleq \{y \in \mathcal{A} \mid y \succ x \wedge AMUB(xy) \ge minu \wedge \text{supp}(xy) \ge mins\}$ and $sCMAP(x) \quad sCMAP(x) \triangleq \{y \in \mathcal{A} \mid AMUB(x \rightarrow y) \ge minu \wedge \text{supp}(x \rightarrow y) \ge mins\}$. The sets $iCMAP(x)$ and $sCMAP(x)$ are respectively candidate items for extensions of any sequence α of which the last item is x, i.e. $lastItem(\alpha) = x$. Then, since $\mathcal{AM}(AMUB)$ and $\mathcal{AM}(supp)$, if $y \notin iCMAP(x)$ (or $y \notin sCMAP(x)$), any extension $\alpha \diamond_i y \diamond \delta$ of $\alpha \diamond_i y$ (or $\alpha \diamond_s y \diamond \delta$ of $\alpha \diamond_s y$) cannot be an FLCHU sequence. *Indeed*, for instance, if $y \notin iCMAP(x)$ and $AMUB(xy) < minu$ or $supp(xy) < mins$, then $AMUB(\alpha \diamond_i y \diamond \delta) \le AMUB(xy) < minu$ or $supp(\alpha \diamond_i y \diamond \delta) \le supp(xy) < mins$, i.e. $y \notin I(x)$ and $\alpha \diamond_i y \diamond \delta$ is not a FLCHU sequence.

Note that the two sets $iCMAP(x)$ and $sCMAP(x)$ are calculated only *once* for any item x of $\mathcal{A}$. Therefore, to speed up mining *FLCHUS* using the $\mathcal{TS}$ *tightening* strategy, *CMAPS* (i.e. $iCMAP(x)$ and $sCMAP(x)$) can be additionally used as follows: if $y \notin iCMAP(x)$ or $y \notin sCMAP(x)$, then the item y should not be put into the $I(\alpha)$ or $S(\alpha)$ sets (such that $lastItem(\alpha) = x$), and to waste time computing the *AMUB* UB of $\alpha \diamond y$, which are used for $I(\alpha)$ or $S(\alpha)$, respectively. In other words, two *tighter* new sets of candidate items for extensions are derived: $I(\alpha) = \{y \in iCMAP(x) \mid y \succ x \wedge AMUB(\alpha \diamond_i y) \ge minu \wedge supp(\alpha \diamond_i y) \ge mins\}$ and $S(\alpha) = \{y \in sCMAP(x) \mid AMUB(\alpha \diamond_s y) \ge minu \wedge supp(\alpha \diamond_s y) \ge mins\}$. Hence, $I(\alpha \diamond_i x) \subseteq I(\alpha) \cap iCMAP(x)$ and $I(\alpha \diamond_s x) \cup S(\alpha \diamond_i x) \cup S(\alpha \diamond_s x) \subseteq S(\alpha) \cap sCMAP(x)$.

For example, for $minu = 40$ and $mins = 3$, after the items b and g are removed from $\mathcal{D}$', consider the item f in $S(<>) = \{a, c, d, e, f\}$. First, we have $iCMAP(c) = \{e\}$ and $sCMAP(c) = \{a, d, f\}$, because $supp(cd) = 0 < supp(cf) = supp(c \rightarrow c) = supp(c \rightarrow e) = 1 < mins$. Besides, $I(d) = \varnothing$ and $S(d) = \{a, c, f\}$. Thus, for $\alpha = d \rightarrow c$ with $lastItem(\alpha) = c$, then $I(\alpha) = \varnothing$ ($\subseteq iCMAP(c) \cap S(d) = \varnothing$). Similarly, for finding $S(\alpha)$, we only need to consider two items in $sCMAP(c) \cap S(d) = \{a, f\}$. Finally, we also have $S(\alpha) = \{a, f\}$, because $AMUB(\alpha \rightarrow a) = AMUB(\alpha \rightarrow f) = 153 \ge minu$ and $supp(\alpha \rightarrow a) = supp(\alpha \rightarrow f) = 3 \ge mins$.

5 The FLCHUSPM algorithm

To efficiently mine the set of *FLCHUS*, a new data structure *CUL* (*Cost-Utility List*) and a novel algorithm named FLCHUSPM (*Frequent Low average Cost High average Utility Sequential Pattern Mining*) is designed.

5.1 A novel data structure

Before presenting the proposed FLCHUSPM algorithm (in Section 5.2), the next paragraphs describe a novel vertical data structure, named *CUL* (*Cost-Utility List*). This structure is designed to store necessary intermediate data during sequence mining and to support the quick calculation of information about the cost and utility of a pattern's extensions as well as their corresponding upper/lower bounds.

5.1.1 *The CUL structure* of a sequence

Given a sequence α and an input q-sequence $\Psi_i' \in \rho(\alpha)$, let c_f be the *cost* of the first occurrence of α in Ψ_i', and for each ending $end \triangleq end\left(\alpha, \Psi_i'\right)$ of α in Ψ_i', let $u_{end} = u\left(\alpha, \Psi_i', end\right) \triangleq \max\left\{u(\alpha') \mid \alpha' \in \mathcal{O}\left(\alpha, \Psi_i'\right) \wedge end\left(\alpha', \Psi_i'\right) = end\right\}$ and $c_{end} = c\left(\alpha, \Psi_i', end\right) \triangleq \max\left\{\left(\alpha'\right) \mid \alpha' \in \mathcal{O}\left(\alpha, \Psi_i'\right) \wedge end\left(\alpha', \Psi_i'\right) = end\right\}$ be respectively the *maximum* utility and the *maximum cost* of α according to end in Ψ_i', $ru_{end} \triangleq u\left(rem\left(\alpha, \Psi_i', end\right)\right)$ be the utility of the *remaining* sequence $rem\left(\alpha, \Psi_i', end\right)$. Furthermore, we denote c_f and the list of tuples $tup(end) \triangleq (end, c_{end}, u_{end}, ru_{end})$ as $tl\left(\alpha, \Psi_i'\right)$ or briefly $tl \triangleq \left(c_f; \left\{tup(end) \mid end = end\left(\alpha, \Psi_i'\right)\right\}\right)$. Without loss generality, it is assumed that the *tl* list *is* arranged in ascending order of *end*. Then, the structure *CUL* of α is defined as $CUL(\alpha) \triangleq \left\{< i, tl\left(\alpha, \Psi_i'\right) > \mid \Psi_i' \in \rho(\alpha)\right\}$. Next, the *input q-sequence utility* vector is defined as $ISU \triangleq \left(u\left(\Psi_i'\right), i = 1..N\right)$. After the QSDB $\mathcal{D}$' is scanned for the first time and then reduced by removing all irrelevant items in $\mathcal{A} \backslash \mathcal{S}$, where $S \triangleq \{x \in \mathcal{A} \mid AMUB(x) \ge minu \wedge supp(x) \ge mins\}$, the *ISU* is

calculated only once. Also, the *CUL* of relevant items is calculated, i.e. $\{CUL(x)|x\in\mathcal{A}\backslash S\}$.

For example, for *minu*=38, *mins*=3, since *supp*(*g*)=1<*supp*(*b*)=2<*mins*, FLCHU sequences cannot contain the items *b* and/or *g*. Thus, the two items are removed from $\mathcal{D}$'. Then, we obtain *ISU*=(132;23;168;159) from the reduced QSDB.

Figure 1 illustrates the *CUL* of relevant items. Consider the calculation of the *CUL* of *d*, *CUL*(*d*). Since *d* first appears in the 1st event of Ψ_3' according to the *q*-itemset (*d*,4,8), then *end*=1, $c_f=c((d,4,8))=4$, $c_{end}=max\{c((d,4,8))\}=4$, $u_{end}=max\{u((d,4,8))\}=8$, $rem(d,\Psi_3',1)=(a,23,7)(c,3,35)(e,12,15)\rightarrow(d,7,22)\rightarrow(a,8,9)(f,3,72)$, and $ru_{end}=160$. We obtain thus the first tuple *tup*(1)=(1,4;8,160). For the second appearance of *d* in Ψ_3' at the ending *end*=3, we have $c_{end}=max\{c((d,7,22))\}=7$, $u_{end}=max\{u((d,7,22))\}=22$ and $ru_{end}=81$. Hence, the second tuple *tup*(3)=(3,7;22,81), so $tl(\alpha,\Psi_3')=\{(1,4;8,160),(3,7;22,81)\}$. Similarly, $CUL(d)\stackrel{def}{=}\{<1,(3;\{(1,3;80,52)\})>,<2,(5;\{(2,5;1,14),(3,1;4,10)\})>,<3,(4;\{(1,4;8,160),(3,7;22,81)\})>,<4,(3;\{(3,3;50,95)\})>\}$. The tuples of the remaining relevant items are calculated in the same way.

Having described the *ISU* and *CUL*(α) of a sequence α, the next paragraphs explain how they can be used to quickly calculate the *support*, *utility* and *cost* information (*supp*, *au*, *ac*, *AMUB*, *FUB*, *FLB*) of α as well as its extensions.

5.1.2 Calculating information about a sequence using its *CUL*

For any sequence α such that $supp(\alpha)\geq m\stackrel{def}{=}mins$, since $\rho(\alpha)=\{\Psi_i'|(i,tl)\in CUL(\alpha)\}$, $\Psi_i'\in\rho(\alpha)\Leftrightarrow(i,tl)\in CUL(\alpha)$, we have $u(\alpha,\Psi_i')=\max\{t.u_{end}\mid t\in tl\}$, $ub(\alpha,\Psi_i')=max\{t.u_{end}+ru_{end}|\ t\in tl\}$, $c(\alpha,\Psi_i')=max\{t.c_{end}|\ t\in tl\}$ and $lb(\alpha,\Psi_i')=c(\alpha_{first}')=tl.c_f$. Hence, for each *end* in (*i*,*tl*), the values *supp*, *au*, *ac*, *AMUB*, the *FUB* UBs, and the *FLB* WUB *of* α *can be simply calculated using the CUL*(α) and *ISU as follows.* First, all values of $(u(\Psi_i'),\Psi_i'\in\rho(\alpha))$ and $(ub(\alpha,\Psi_i'),\Psi_i'\in\rho(\alpha))$ are sorted in *descending* order, and values of $(tl.c_f,(i,tl)\in CUL(\alpha))$ are sorted in *ascending* order. Then, the calculations are:

(*i*). $s=supp(\alpha)=|CUL(\alpha)|$, $au(\alpha)=\frac{1}{s}\sum_{(i,tl)\in CUL(\alpha)}u(\alpha,\Psi_i')$ and $ac(\alpha)=\frac{1}{s}\sum_{(i,tl)\in CUL(\alpha)}c(\alpha,\Psi_i')$.

(*ii*).

$$AMUB(\alpha)=\frac{1}{m}\sum_{m\ first\ largest\ values\ of\ (u(\Psi_i'):(i,tl)\in CUL(\alpha))}u(\Psi_i'),$$

$$FUB(\alpha)=\frac{1}{m}\sum_{m\ first\ largest\ values\ of\ (ub(\alpha,\Psi_i'):(i,tl)\in CUL(\alpha))}ub(\alpha,\Psi_i').$$

(*iii*).

$$FLB(\alpha)=\frac{1}{m}\sum_{m\ first\ smallest\ values\ of\ (tl.c_f,(i,tl)\in CUL(\alpha))}tl.c_f..$$

For example, for *minu*=38, *m*=*mins*=3, α=*d*, $\rho(\alpha)=\mathcal{D}$', we have $supp(\alpha)=|CUL(d)|=|\rho(\alpha)|=4\geq m$, $u(\alpha,\Psi_1')=80$, $u(\alpha,\Psi_2')=max\{1,4\}=4$, $u(\alpha,\Psi_3')=max\{8,22\}=22$, $u(\alpha,\Psi_4')=50$, so $au(\alpha)=(80+4+22+50)/4=39$. Since *ISU*=(132; 23; 168; 159) and $\rho(\alpha)=\mathcal{D}$' and after sorting $(ub(\alpha,\Psi_i'),i=1..4)=(132;15;168;145)$ and *ISU* in *descending* order, and taking the 3 (=*m*) first (*largest*) values, we obtain (168; 159; 132; 23), (168; 145; 132; 15), and $AMUB(\alpha)=(168+159+132)/3=153$, $FUB(\alpha)=(168+145+132)/3=445/3\simeq 148.3$. On the other hand, $c(\alpha,\Psi_1')=3$, $c(\alpha,\Psi_2')=max\{5,1\}=5$, $c(\alpha,\Psi_3')=max\{4,7\}=7$, $c(\alpha,\Psi_4')=3$, so $ac(\alpha)=(3+5+7+3)/4=4.5$, and after sorting $(tl.c_f|(i,tl)\in CUL(\alpha))=(3;5;4;3)$ in *ascending* order, we have (3;3;4;5) and by taking the three (=*m*) first (*smallest*) values, we obtain $FLB(\alpha)=(3+3+4)/3=10/3$. Therefore, $au(\alpha)<FUB(\alpha)<AMUB(\alpha)$, and $ac(\alpha)>FLB(\alpha)$.

Fig. 1 The *CUL* of relevant items

a	
Sid	c_f; (*end*, c_{end}; u_{end}, ru_{end})
1	10; (2, 10; 5, 47), (3, 7; 3, 4)
2	3; (4, 3; 1, 9)
3	23; (2, 23; 7, 153), (4, 8; 9, 72),
4	13; (2, 13; 3, 145), (4, 4; 5, 90), (5, 14; 4, 46)

c	
Sid	c_f; (*end*, c_{end}; u_{end}, ru_{end})
1	2; (2, 2; 40, 7)
2	7; (1, 7; 2, 21)
3	3; (2, 3; 35, 118)
4	21; (1, 21; 5, 154), (4, 2; 40, 50), (5, 1; 10, 36)

d	
Sid	c_f; (*end*, c_{end}; u_{end}, ru_{end})
1	3; (1, 3; 80, 52)
2	5; (2, 5; 1, 14), (3, 1; 4, 10)
3	4; (1, 4; 8, 160), (3, 7; 22, 81)
4	3; (3, 3; 50, 95)

e	
Sid	c_f; (*end*, c_{end}; u_{end}, ru_{end})
1	4; (4, 4; 1, 0)
2	9; (1, 9; 6, 15)
3	12; (2, 12; 15, 103)
4	24; (1, 24; 6, 148)

f	
Sid	c_f; (*end*, c_{end}; u_{end}, ru_{end})
1	1; (3, 1; 3, 1)
2	2; (4, 2; 9, 0)
3	3; (4, 3; 72, 0)
4	2; (5, 2; 36, 0)

5.1.3 Calculating the *CUL* of extensions

This subsection explains how to *quickly calculate the CUL of the* i– *and* s–*extensions of a sequence* α *with any item* y based on $CUL(\alpha)$ and $CUL(y)$. For each element $(i, c_f; tl)$ of $CUL(\alpha)$, we denote $CUL(\alpha)$. $SID \stackrel{def}{=} \{i \mid (i, c_f; tl) \in CUL(\alpha)\}$, $c_f(\alpha) = c_f$ and any tuple in the *tuple list tl* as $t(\alpha, i)$ or briefly $t(\alpha) = (end, c_{end}; u_{end}, remu_{end})$.

***Calculating the CUL* of extensions** To create the $CUL(\beta)$ of the *i*-extension $\beta = \alpha \diamond_i y$ (or *s*-extension $\beta = \alpha \diamond_s y$), for each $i \in CUL(\alpha)$. $SID \cap CUL(y)$. SID, $t(y)$ $(= t(y, i) \in tl(y, \Psi_i'))$ and each *fixed* $t(y)$. end, we consider all $t(\alpha) \in tl(\alpha, \Psi_i')$ such that $t(\alpha)$. $end = t(y)$. end (or $t(\alpha)$. $end < t(y)$. end, respectively). Then, we create the corresponding tuple $t(\beta)$ as follows: $t(\beta)$. $end = t(y)$. end, $t(\beta)$. $remu_{end}$, $= t(y)$. $remu_{end}$, $t(\beta)$. $u_{end} = t(\alpha)$. $u_{end} + t(y)$. u_{end} and $t(\beta)$. $c_{end} = t(\alpha)$. $c_{end} + t(y)$. c_{end} (or $t(\beta)$. $u_{end} = max\{t(\alpha).\ u_{end} + t(y).\ u_{end} \mid t(\alpha).\ end < t(y).\ end\}$ and $t(\beta).\ c_{end} = max\{t(\alpha).\ c_{end} + t(y).\ c_{end} \mid t(\alpha).\ end < t(y).\ end\}$, respectively). Moreover, for the *first* $t(y)$. end and $t(\alpha)$. end such that $t(\alpha)$. $end = t(y)$. end (or $t(\alpha)$. $end < t(y)$. end, respectively), then $c_f(\beta) = t(\alpha)$. $c_{end} + t(y)$. c_{end}. Finally, we add $c_f(\beta)$ and the new tuple $t(\beta) = (t(\beta).\ end, t(\beta).\ c_{end}; t(\beta).\ u_{end}, t(\beta).\ remu_{end})$ to the *tl* of the element $(i, c_f(\beta); tl)$ of $CUL(\beta)$.

For example, consider that $\alpha = a$ and $y = c$. The *CUL* of the *i*-extension $\beta = \alpha \diamond_i y = ac$ is shown in Fig. 2a. There exist four $SID \in CUL(a)$. $SID \cap CUL(c)$. $SID = \{1, 2, 3, 4\}$. For instance, according to $SID = i = 1$, there exists one pair of tuples $(t(a), t(c))$ $(=((2, 10; 5, 47), (2, 2; 40, 7)))$ such that $t(a)$. $end = t(c)$. end. Then, $t(\beta)$. $end = t(c)$. $end = 2$, $t(\beta)$. $c_{end} = 10 + 2 = 12$, $t(\beta)$. $u_{end} = 5 + 40 = 45$, $t(\beta)$. $ru_{end} = t(c)$. $ru_{end} = 7$. Therefore, one tuple list $< 1, tl > = < 1, \{(2, 12; 45, 7)\} >$ of $CUL(ac)$ is obtained and its remaining tuple lists in Fig. 2a are calculated in the same way.

Similarly, the *CUL* of the *s*-extension $\gamma = \alpha \diamond_s y = a \rightarrow c$ is shown in Fig. 2b. There exist four $SID \in CUL(\alpha)$. $SID \cap CUL(c)$. $SID = \{1, 2, 3, 4\}$. However, for $SID \in \{1, 2, 3\}$, there exist no pair $(t(a), t(c))$ such that $t(a)$. $end < t(c)$. end. For $SID = i = 4$ and $t(c)$. $end = 4$ (of $t(c) = (4, 2; 40, 50)$). There uniquely exists $t(a) = (2, 13; 3, 145)$ such that $t(a)$. $end = 2 < t(c)$. end, then $t(\gamma)$. $end = t(c)$. $end = 4$, $t(\gamma)$. $c_{end} = 13$.

$+2 = 15$, $t(\gamma)$. $u_{end} = 3 + 40 = 43$, and $t(\gamma)$. $ru_{end} = t(c)$. $ru_{end} = 36$. For another $t(c)$. $end = 5$ of $t(c) = (5, 1; 10, 36)$, there exist two pairs $t(a) \in \{(2, 13; 3, 145), (4, 4; 5, 90)\}$ such that $t(a)$. $end < t(c)$. end. Thus, $t(\gamma)$. $end = t(c)$. $end = 5$, $t(\gamma)$. $c_{end} = max\{13 + 1, 4 + 1\} = 14$, $t(\gamma)$. $u_{end} = max\{3 + 10, 5 + 10\} = 15$, $t(\gamma)$. $ru_{end} = t(c)$. $ru_{end} = 36$. Thus, we obtain $(\gamma) = \{< 4, \{(4, 15; 43, 50), (5, 14; 15, 36)\} >\}$.

5.2 The algorithm

Based on the theoretical results and *CUL* data structure presented in Sections 3 and 4.1, the following algorithm FLCHUSPM is designed to solve the *FLCHUSM* problem. To our best knowledge, FLCHUSPM is the first algorithm for mining *FLCHUS*. Since *supp*, the *FUB* and *AMUB* UBs, and the *FLB* LB have different pruning properties, applying all of them into FLCHUSPM is necessary to further reduce the search space. Moreover, it only requires scanning a QSDB twice.

Figure 3 shows the pseudo-code of FLCHUSPM. The algorithm takes as input a QSDB $\mathcal{D}'$ and some user-defined *minu*, *maxc* and *mins* thresholds. The three $\mathcal{RedS}$, $\mathcal{WPS}$ and *CMAPS* strategies are applied into FLCHUSPM at the first level of the prefix search tree. First, the QSDB $\mathcal{D}'$ is scanned to compute the set $S = I(<>) = S(<>) \stackrel{def}{=} \{x \in \mathcal{A} \mid AMUB(x) \geq minu \wedge \text{supp}(x) \geq mins\}$ of candidate items to form FLCHUs according to the *AMUB* and *supp* values (line 1). Then, the algorithm removes from $\mathcal{D}'$ all irrelevant items in $\mathcal{A} \backslash S$ and updates the *AMUB* and *FUB* upper bounds of all relevant items in *S* (lines 2–3). Next, it computes once the *iCMAP* and *sCMAP* sets (of candidate items for *i*- and *s*-extensions) of only relevant items in *S* by scanning the reduced database $\mathcal{D}'$ (line 4). Finally, the algorithm calls the recursive procedure SearchFLCHUS for each item $x \in S$ (line 5).

The **SearchFLCHUS** procedure, shown in Fig. 4, takes as input a sequence α, and two sets *I* and *S* of items to be used for extending the α's prefix by *i*- and *s*-extensions. First, the procedure applies the depth pruning $\mathcal{DPS}$ strategy (line 1), if the depth pruning condition $DepthPC(\alpha)$ holds, $branch(\alpha)$ is immediately pruned. Next, if $au(\alpha) \geq minu$ and $ac(\alpha) \leq maxc$, the FLCHU sequence α is output (line 2). Then, the width pruning and tightening strategies, ($\mathcal{WPS}$, $\mathcal{TS}$ and *CMAPS*), are applied for extensions of α (lines 3–8). Finally, SearchFLCHUS recursively calls itself for each item in the two $I(\alpha)$ and $S(\alpha)$ sets of candidate items for *i*- and *s*-extensions of α (lines 9–10).

Note that the correctness of FLCHUSPM is guaranteed by Theorems 1–3 and FLCHUSPM discovers the complete set of FLCHU sequences in the *QSDB* $\mathcal{D}'$.

Now, we illustrate some main steps of FLCHUSPM for the running example with $minu = 40$, $maxc = 6.6$ and $mins = 3$ (Fig. 5). Each node of the prefix search tree is represented in the brief form $\alpha^{AMUB;FUB,Flb}_{au,ac;supp}$. As discussed in Remark b, since $supp(g) = 1 < supp(b) = 2 < mins$, not only the two branches $branch(b)$ and $branch(g)$ are widely pruned, but we can also remove two irrelevant items *b* and *g* from $\mathcal{D}'$ (line 1–2 in Fig. 3) and update all UBs of all remaining items in $S = \{a, c, d, e, f\}$ as mentioned in Remarks a. Next, in lines 4–5, the $iCMAP(x)$ and $sCMAP(x)$ are calculated (see Remarks b) and the procedure SearchFLCHUS(x, S, S) is called for all items x in S.

For example, in Fig. 4, for any item $x \in \{a, e, f\}$, $branch(x)$ is *deeply* pruned because $maxc < Flb(e) < Flb(a)$ and $FUB(f) < minu$ (line 1). For the item *d*, since $au(d) < minu$, it is not a FLCHU sequence (line 2). By lines 3–8, $I(d) = iCMAP(d) = \varnothing$

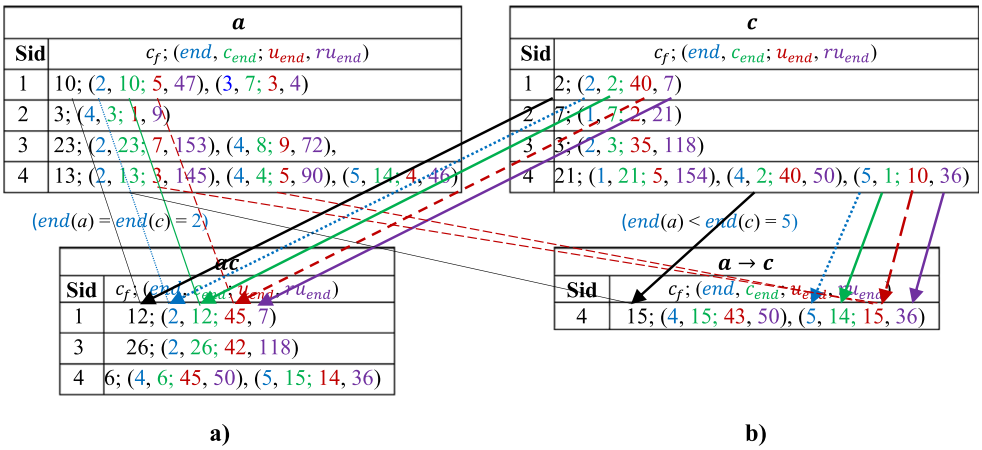


Fig. 2 The *CUL* of $a \diamond_i c$ and $a \diamond_s c$

and $S(d)=sCMAP(d)=\{a,c,f\}$ because $supp(de)=supp(df)=0<supp(d\rightarrow d)=supp(d\rightarrow e)=2<mins$, i.e. $branch(d\rightarrow d)$ and $branch(d\rightarrow e)$ are *widely* pruned, and calculating bounds on $d\rightarrow d$ and $d\rightarrow e$ is not necessary. Next, line 9 is skipped because $I(d)=\emptyset$. In line 10, the procedure SearchFLCHUS($d\diamond_s y$, $I(d)$, $S(d)$) is recursively invoked for all items y in $S(d)$. For $a\in S(d)$, since $FUB(d\rightarrow a)<minu$, $d\rightarrow a$ is *deeply* pruned. For the two remaining items c, $f\in S(d)$, we have two corresponding FLCHU sequences $\alpha\in\{d\rightarrow c, d\rightarrow f\}$, because $au(\alpha)\geq minu$, $ac(\alpha)\leq maxc$ and $supp(\alpha)\geq mins$. Furthermore, $S(d\rightarrow f)=I(d\rightarrow f)=\emptyset\subseteq S(d)$, i.e. all sequences of the $branch(d\rightarrow f)$ will not be considered. This process is similarly applied for the two nodes c and $d\rightarrow c$ (see Remarks b), but no other LCHU sequence is obtained, as shown in Fig. 5. Finally, it is found that $FLCHUS=\{d\rightarrow c, d\rightarrow f\}$.

6 Experimental evaluation

To evaluate the performance of the proposed algorithm, experiments were performed on a computer equipped with an Intel(R) Core ™ 3.0 GHz i5–2320 CPU with 8 GB of memory, running Windows 8.1. The algorithm was implemented in Java 1.8 and experiments were conducted on two real-life SDBs named BMS and Snake, and three synthetic SDBs named D4C7T5N5S6I4, D0.5C10T15N2S6I4 and D5C10T5N5S6I4, which have been generated by the IBM Quest data generator (obtained from [41]). Parameters of this generator are described in Table 2. To obtain QSDBs from the SDBs, utilities and costs of items were generated respectively in the [10, 100] and [1, 70] intervals using a log-normal distribution.

The database characteristics are shown in Table 3. These datasets were selected to evaluate the algorithm's performance as they are public benchmark datasets used in many sequential pattern mining papers, and they have varied characteristic, which allow to observe the algorithm's performance in different context. The three D4C7T5N5S6I4, D5C10T5N5S6I4 and BMS QSDBs are *sparse*, D0.5C10T20N2S6I4 is dense, and Snake is very *dense*. The database densities ($\delta \stackrel{def}{=} T/N$) are 0.1, 0.1, 0.2, 1 and 5 % respectively. BMS and D0.5C10T20N2S6I4 contain many very long sequences, while D5C10T5N5S6I4 is used to test the scalability of the proposed algorithm. Note that FLCHUSPM is the first algorithm for solving the new *FLCHUSM* problem, where the average utility (or cost) of a sequential pattern is calculated as its utility (or cost, respectively) divided by its support instead of by its length as used in the HAUSM problem for mining traditional high average utility sequences ([23, 37, 38]). Besides, the proposed problem is also totally different from the HAUIM problem on quantitative transaction databases ([21, 22, 42]), and the problems considered in [39, 40] as explained in the Discussion of Section 3. Because of this and that the previous problems and the FLCHUSM problem have different goals as well as different parameters, it would be unfair to compare the performance of FLCHUSPM with

Fig. 3 The FLCHUSPM algorithm for mining *FLCHUS*

FLCHUSPM($\mathcal{D}'$)
. *Input*: QSDB $\mathcal{D}'$.
. *Output*: the set $\mathcal{FLCHUS}$ of all FLCHU sequences.
1. Scan $\mathcal{D}'$ once to find $S=\{x\in\mathcal{A}\,|AMUB(x)\geq minu\wedge supp(x)\geq mins\}$;
2. **Remove** from $\mathcal{D}'$ all *irrelevant* items in $\mathcal{A}\backslash S$; // *using* $\mathcal{RedS}$ *strategy*
3. **Update** bounds of all *remaining* items in S; // *using* $\mathcal{WPS}$ *strategy*
4. Calculate $iCMAP$ and $sCMAP$ for all items in S; // *using* $\mathcal{CMAPS}$ *strategy*
5. **for each** item $x\in S$ **do SearchFLCHUS**(x, S, S);

Fig. 4 The SearchFLCHUS procedure

SearchFLCHUS(α, I, S)
. *Input*: a sequence α, two sets of candidate items I and S to extend the prefix of α.
. *Output*: α if it is a FLCHU sequence.
1. **if** ($FUB(\alpha) < minu \vee FLB(\alpha) > maxc$) **then return**; // *using $\mathcal{DPS}$ strategy*
2. **if** ($au(\alpha) \geq minu \wedge ac(\alpha) \leq maxc$) **then** output the FLCHU sequence α
3. $I(\alpha) = S(\alpha) = \emptyset$; // *candidate item sets for i- and s-extensions of α*
   // *applying $\mathcal{CMAPS}$ and $\mathcal{TS}$ in $\mathcal{WPS}$ strategies:*
4. $x = lastItem(\alpha)$;
5. **for each** item $y \in I$ such that $y \succ x$ **do**
6. **if** ($y \in iCMAP(x) \wedge AMUB(\alpha \diamond_i y) \geq minu \wedge supp(\alpha \diamond_i y) \geq mins$) **then** $I(\alpha) = I(\alpha) \cup \{y\}$;
7. **for each** item $y \in S$ **do**
8. **if** ($y \in sCMAP(x) \wedge AMUB(\alpha \diamond_s y) \geq minu \wedge supp(\alpha \diamond_s y) \geq mins$) **then** $S(\alpha) = S(\alpha) \cup \{y\}$;
9. **for each** item $y \in I(\alpha)$ **do SearchFLCHUS**($\alpha \diamond_i y$, $I(\alpha)$, $S(\alpha)$);
10. **for each** item $y \in S(\alpha)$ **do SearchFLCHUS**($\alpha \diamond_s y$, $S(\alpha)$, $S(\alpha)$);

the algorithms of the aforementioned papers. Thus, to evaluate the performance of FLCHUSPM, we instead compared different versions of FLCHUSPM, while varying parameter values and dataset characteristics.

The four above datasets are used for performance experiment. A fifth dataset obtained from the Deeds e-learning platform will be discussed in Section 6.3 to evaluate the quality of the discovered patterns in a real application.

Experimental results, which are presented next show that the runtime of the algorithm depends on the number of extensions performed and increases when the *mins* or *minu* thresholds are decreased or the *maxc* threshold is increased. In the following, the *mins*, *minu* and *maxc* parameters will be denoted briefly as *ms*, *mu* and *mc*.

6.1 Influence of pruning strategies

An experiment was first performed to assess the influence of pruning strategies on FLCHUSPM's performance. We first study the influence of the width pruning $\mathcal{WPS}$ strategy (including $\mathcal{RedS}$ *reducing*, $\mathcal{TS}$ and *CMAPS* tightening strategies) based on the support (*supp*) and/or *AMUB* UB, and the influence of the depth pruning $\mathcal{DPS}$ strategy based on the *FUB* UB and the *Flb* LB. To assess the pruning effect (*PE*) of the proposed pruning strategies, the performance of FLCHUSPM is compared in terms of runtime and number of extensions in the three following cases: (1) only applying the $\mathcal{WPS}$ strategy (or briefly **WPS**), (2) only using the $\mathcal{DPS}$ strategy (**DPS**) and (3) using both the $\mathcal{DPS}$ and $\mathcal{WPS}$ strategies (**All**). Note that, the $\mathcal{WPS}$ strategy based on the traditional *supp* measure is also applied in the second case (DPS). The ranges of threshold values used in the experiments were selected such that the cardinality of *FLCHUS* varied from 1 to large values. For example, Table 4 indicates the cardinality of *FLCHUS* when fixing *mins* = 0.1% and *minu* = 0.11%, for different *maxc* values. Results of the experiment are shown in Fig. 6. It is found that in all cases, (All) outperforms (DPS) and (WPS).

When fixing *ms*=0.1%, *mu*=0.11% and increasing *mc*, the runtime of (All) on the BMS QSDB is on average 6 and 12 times faster than (DPS) and (WPS), respectively. When *mu*=0.1%, *mc*=0.35% and *ms* is decreased, the runtime gaps between (All) and the runtimes of (DPS) and (WPS) significantly increase. It is found that (All) is on average 20 and 30 times faster than (DPS) and (WPS). When decreasing *mu* while using much smaller values such as *ms* = 0.0016% and larger

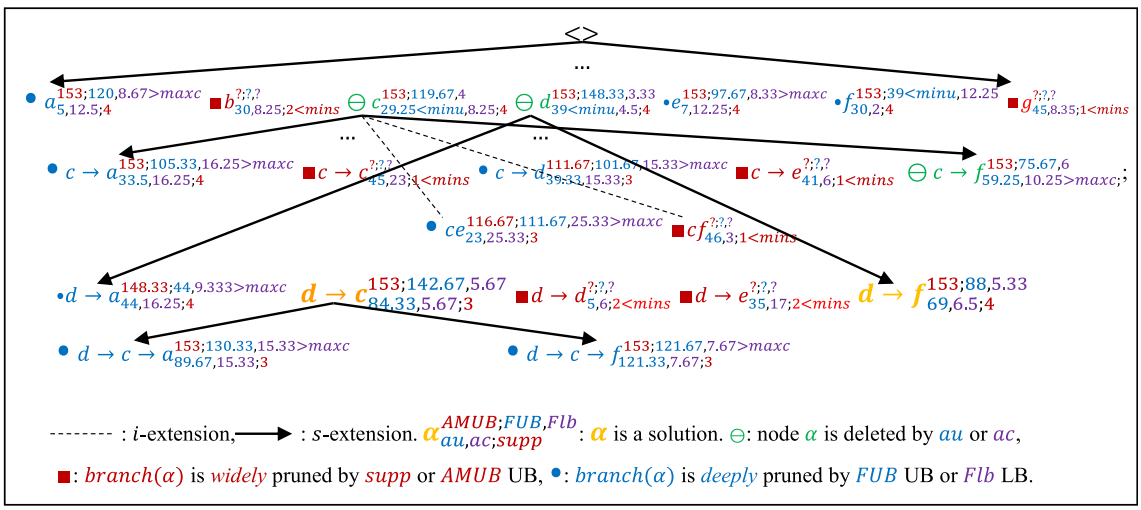


Fig. 5 Process of mining *FLCHUS* with *minu*=40, *maxc*=6.6 and *mins*=3

Table 2 Parameters of the IBM quest synthetic data generator

Parameters	Meaning
D	Number of sequences (in thousands) in the database
C	Average number of itemsets per sequence
T	Average number of items per itemset
N	Number of different items (in thousands) in the database
S	Average number of itemsets in maximal sequences
I	Average number of items in maximal sequences

values such as $mc = 1\%$, (All) and (DPS) are much faster than (WPS). For example, for $mu = 99\%$, the execution times of (All) and (DPS) are respectively 20 and 24 s, but (WPS) does not terminate after three hours. Similar observations are made for the other QSDBs. In terms of number of extensions, the same trends are observed. On the overall, results show that the proposed pruning techniques are effective and can greatly reduce the algorithm's runtime.

6.2 Influence of the thresholds on performance

The second experiment was conducted on the sparse D4C7T5N5S6I4 QSDB and the dense Snake and D0.5C10T20N2S6I4 QSDBs. The influence of the *ms*, *mc* and *mu* thresholds on runtime and number of extensions was evaluated when fixing the two remaining thresholds. Two versions of the FLCHUSPM algorithm are compared: (**All**) and *baseline* algorithm (**BAS**) where only the $\mathcal{WPS}$ (*supp*) strategy based on *supp* is applied, that is not using the $\mathcal{WPS}$ (*AMUB*) and $\mathcal{DPS}$ (*FUB*, *Flb*) strategies, which relied on the *AMUB* and *FUB* UBs, and the *Flb* LB.

Results are shown in Fig. 7a-c, where the D4C7T5N5S6I4 and D0.5C10T20N2S6I4 datasets are briefly denoted as D4C7 and D0.5C10, respectively. It is observed that the runtime gap between FLCHUSPM and BAS increases when *ms* and *mc* are decreased or when *mu* is increased. For example, when *ms* is decreased (and *mu* and *mc* are fixed), the smaller the value of *ms* is, the larger the runtime gap between FLCHUSPM and BAS is. The reason is that the wide pruning $\mathcal{WPS}$ (*supp*) effect is gradually weaker, so the pruning effects of the $\mathcal{WPS}$ (*AMUB*) and $\mathcal{DPS}$ (*FUB*, *Flb*) strategies becomes more significant.

Similar trends have been observed for the three datasets. It is observed in Fig. 7a (or b, c) that FLCHUSPM is faster than BAS by on average about 5, 30 and 37 (or 19, 12 and 4; or 3, 5 and 3) times on Snake (or D4C7T5N5S6I4, D0.5C10T20N2S6I4, respectively).

In the following experiment, we aim to verify part b. of the Discussion in Section 3 using Snake, a large and dense real-life QSDB. We compare the number #*FLCHUS* of mined patterns using the pair of functions (ac, au) proposed in this manuscript with that #*FLCHUS_Old* using for example the pair (ac_{first}, au_s) as presented in [39, 40] by varying each parameter while fixing the two remaining parameters as shown in Fig. 8.a - c. It is observed that the gap between #*FLCHUS_Old* and #*FLCHUS* quickly increases when decreasing *mins* and *minu*, or increasing *maxc*. For example, for the case a, #*FLCHUS_Old* is 2,814 times on average larger than #*FLCHUS*. Thus, using (ac_{first}, au_s) (and similarly (ac_{last}, au_s)) can result in difficulty for users due to the very large final set.

6.3 Analysis of the benefits of considering the cost factor to find patterns

This section aims at evaluating the importance of considering the cost of items to find patterns. The proposed task of *FLCHUSM* is more general than that of *FHUSM* studied in prior work. The latter consists of mining the set *FHUS* of frequent and high utility sequences (FHUSs) and does not consider the cost factor as proposed in this paper. It is clear that these FHUSs can be found by the proposed FHUSPM algorithm that is FLCHUSPM if *maxc* is set to the maximum threshold $maxCSeq \triangleq max\{c(\Psi') | \Psi' \in \mathcal{D}'\}$.

There are three expected benefits of considering the cost factor: (1) reducing the number of patterns, (2) improving the performance and (3) finding more interesting patterns. To check the two first potential benefits, two real-life synthetic QSDBs were used, namely Snake and D4C7. As shown in Fig. 9, the runtime and cardinality of *FLCHUS* mined by

Table 3 Characteristics of databases

Database	#sequences	#distinct items	Avg. seq. length	(min; max) length	Type of data
BMS	59,601	497	2.5	1; 267	click stream
Snake	163	20	60.6	57; 61	protein sequence
D4C7T5N5S6I4	4000	5000	28.7	1; 133	Synthetic
D0.5C10T20N2S6I4	500	2000	163.6	8; 468	Synthetic
D5C10T5N5S6I4	5000	5000	42.9	2; 186	Synthetic

Table 4 The cardinality of *FLCHUS* on BMS (*mins*=0.1%, *minu*=0.11%)

maxc (%)	0.3	0.6	0.9	1.2	2.1
#*FLCHUS*	1	347	1895	3234	3989

FLCHUSPM are much less than the runtime and number of patterns found by FHUSPM. The reason is that FHUSPM considers and mines many uninteresting FHUSs that may have a high cost. Besides, the condition ($Flb(\alpha) > maxc$) in the depth pruning *DepthPC* condition used in FLCHUSPM allows to reduce the search space and to accelerate the process of mining *FLCHUS*. Because the set *FHUS* is larger, it can also be more difficult to analyze by users.

The third potential benefit of considering the cost factor is analyzed in the rest of this section. The task of *FLCHUSM* is applied on data collected from a real-life application, the Deeds e-learning environment. The goal is to verify if patterns found by FLCHUSPM can reveal interesting information compared to those found by FHUSPM, which does not take the cost factor into account

The data from Deeds is publicly available online at https://archive.ics.uci.edu/ml/datasets/ Data+for+ Software +Engineering+Teamwork+Assessment+in+Education+ Setting, and was collected by Vahdat et al. [43]. The Deeds platform was used by 115 students learning digital electronics. The platform recorded various activities done by each learner such as reading course content and solving problems of various difficulty levels to assess their knowledge and skills. The data of each student is a sequence of learning sessions (modules), which can be selected by the learner and performed in different orders. In a session, a student performs a sequence of learning events that have a start time and end time. Moreover, scores are given to the student for each learning session, and a grade is given for the final exam for the whole course. Six learning sessions are available, and each learner is not required to complete all of them nor to follow a strict order. There are fifteen different event types. In the context of this application, the aim is to see if interesting patterns can be found indicating time-effective ways of studying (the time spent is the cost and must not be larger than a given maximum *maxc* threshold) that allows to obtain a high score (the score is the utility and must not be less than a minimum *minu* threshold). For example, a student could be interested in finding patterns that require on average less than 1 h and are associated with obtaining an average score of at least 80%.

This paper proposes to find frequent sequential patterns, which have a high average utility but with a low average cost. Since most sequential pattern mining algorithms do not consider the cost, they can output many patterns that have a high average cost. This is illustrated in Table 5 by comparing patterns found by FLCHUSPM (with *mins* = 1, *minu* = 4.5 and *maxc*=37) with those discovered by FHUSPM (with *mins*=1,

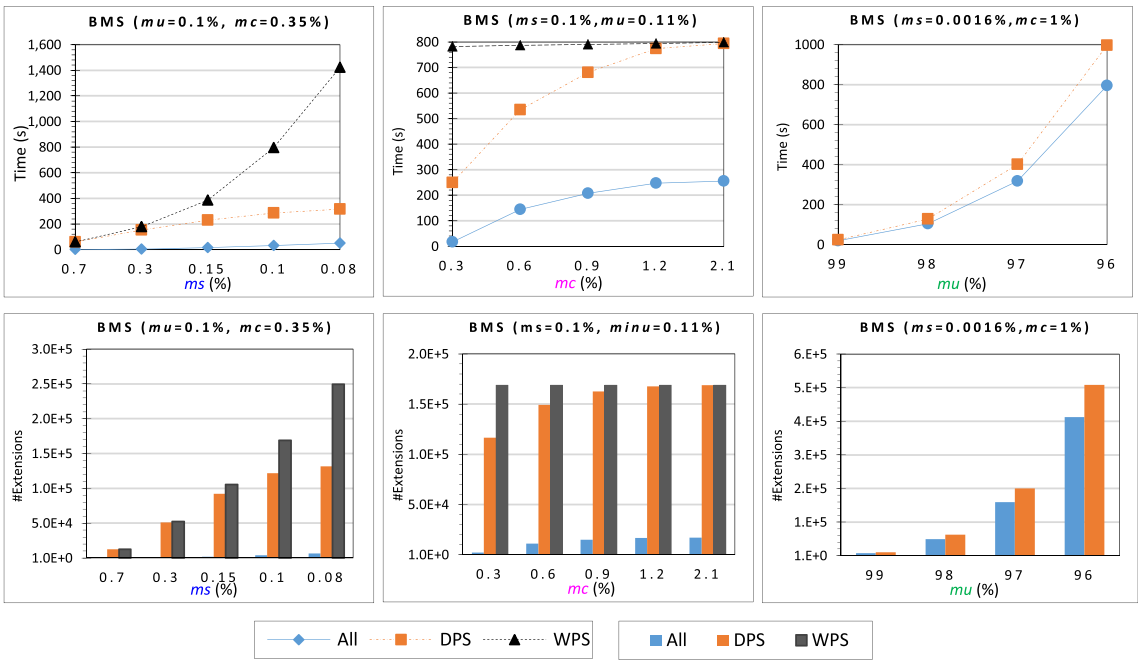
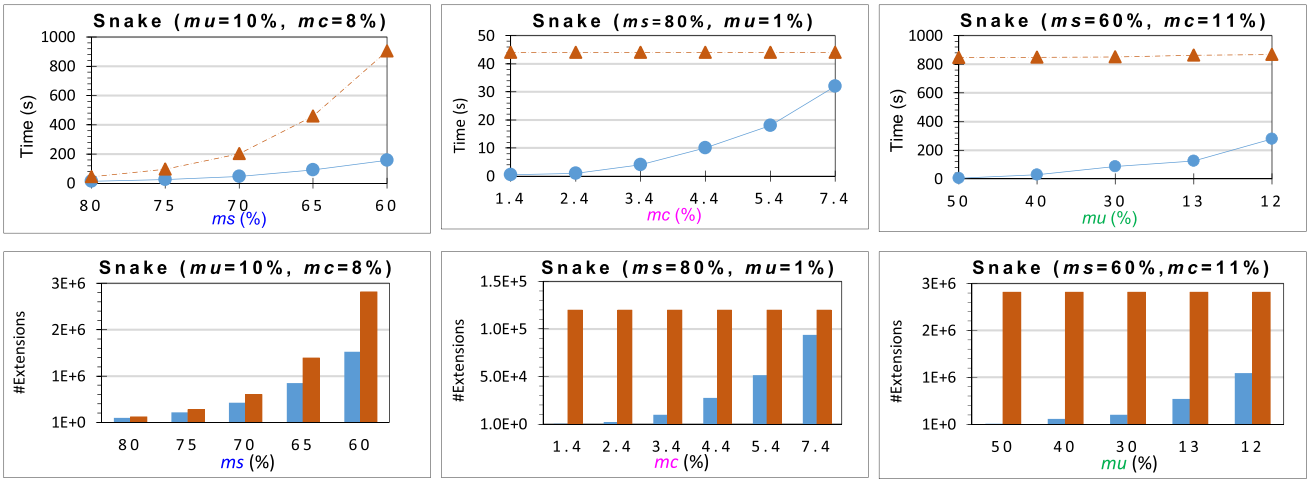
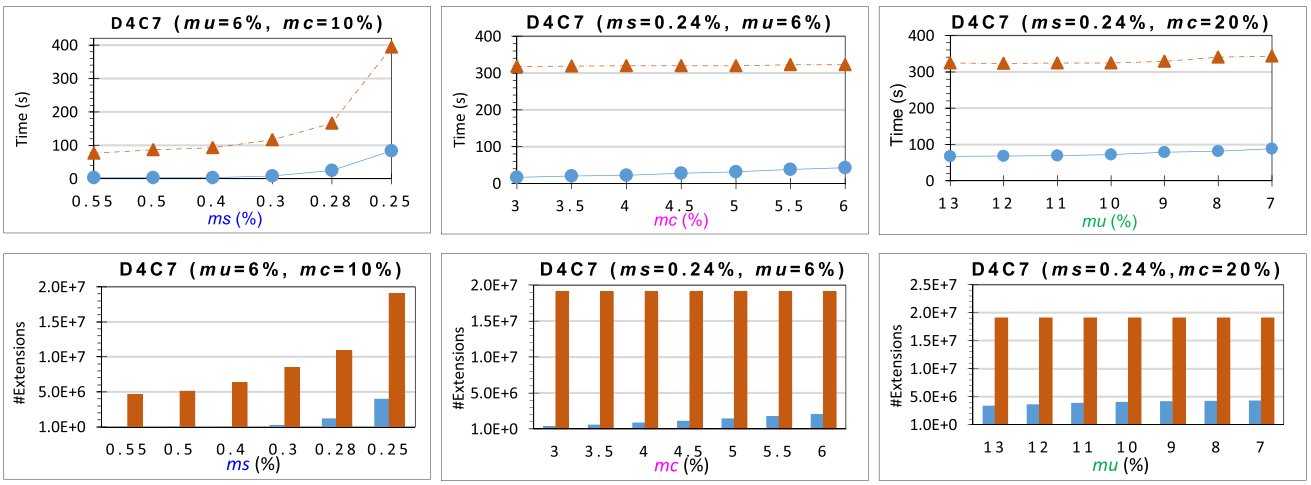


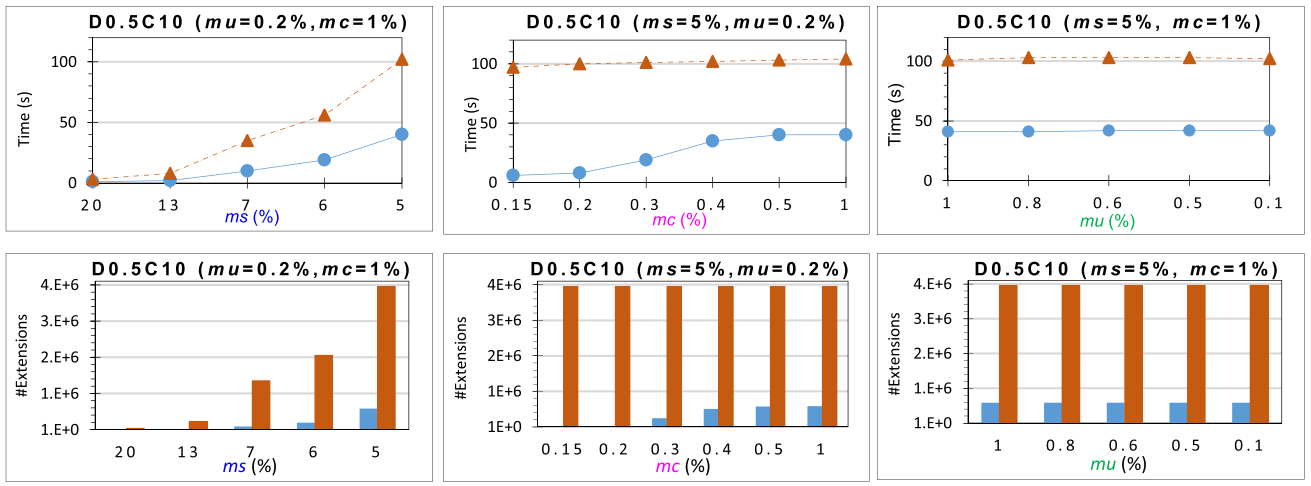
Fig. 6 Evaluating the influence of pruning strategies on performance



a) Results for the Snake dataset.



b) Results for the D4C7T5N5S6I4 dataset.

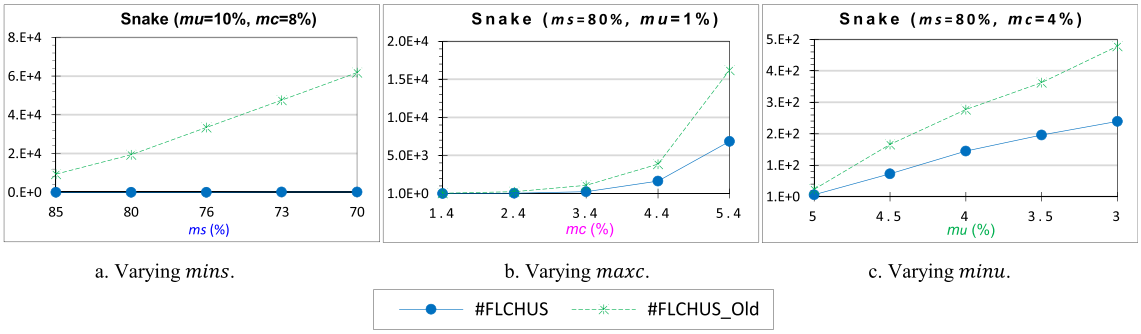


c) Results for the D0.5C10T20N2S6I4 dataset.



Fig. 7. Influence of thresholds on performance.

Fig. 7 Influence of thresholds on performance



a. Varying *mins*. b. Varying *maxc*. c. Varying *minu*.

Fig. 8 Comparison of the number of mined patterns

minu = 4.5 and *maxc* is set to the maximum cost threshold, *maxCSeq* = 495) on the session 5 of the Deeds e-learning dataset. Table 5 shows all of the six patterns found by FLCHUSPM (white color) and some (of the 45,331) patterns found by FHUSPM (gray color). It is observed that for the same average utility, FHUSPM found many patterns that have much higher average cost compared to those found by FLCHUSPM. For example, the two first lines of Table 5 show patterns both having an average utility of 10 but having average costs of 24 and 111, respectively. Besides, there are still many other frequent sequential patterns found by FHUSPM that have an average utility of 10 but their average costs are much higher. For example, beside the pattern found by FHUSPM in the second line, another pattern is *Deeds_Es_5_1→Study_Es_5_1→Deeds_Es_5_1→Study_Es_5_1→Deeds_Es_5_1→Study_Es_5_1→Deeds_Es_5_1→Study_Es_5_1→Deeds_Es_5_1→Study_Es_5_2→Deeds_Es_5_3→Study_Es_5_3→Study_Es_5_4→Deeds_Es_5_4→Study_Es_5_4*. Its average cost (288) is too high (compared to 24). The reason why FHUSPM extracts the first pattern is that only the frequency and utility criteria are evaluated by FHUSPM to select patterns. On overall, this table indicates that patterns having a high average utility and/or high support are not necessarily effective in term of cost. This shows the importance of explicitly considering both cost and utility in a pattern mining for such applications, as proposed in the paper.

6.4 Scalability

To assess the scalability of the FLCHUSPM algorithm for the case (All), the algorithm was applied on a synthetic dataset, named QSDB D5C10T5N5S6I4, which was generated using the IBM Quest Synthetic Data generator [41] according to the

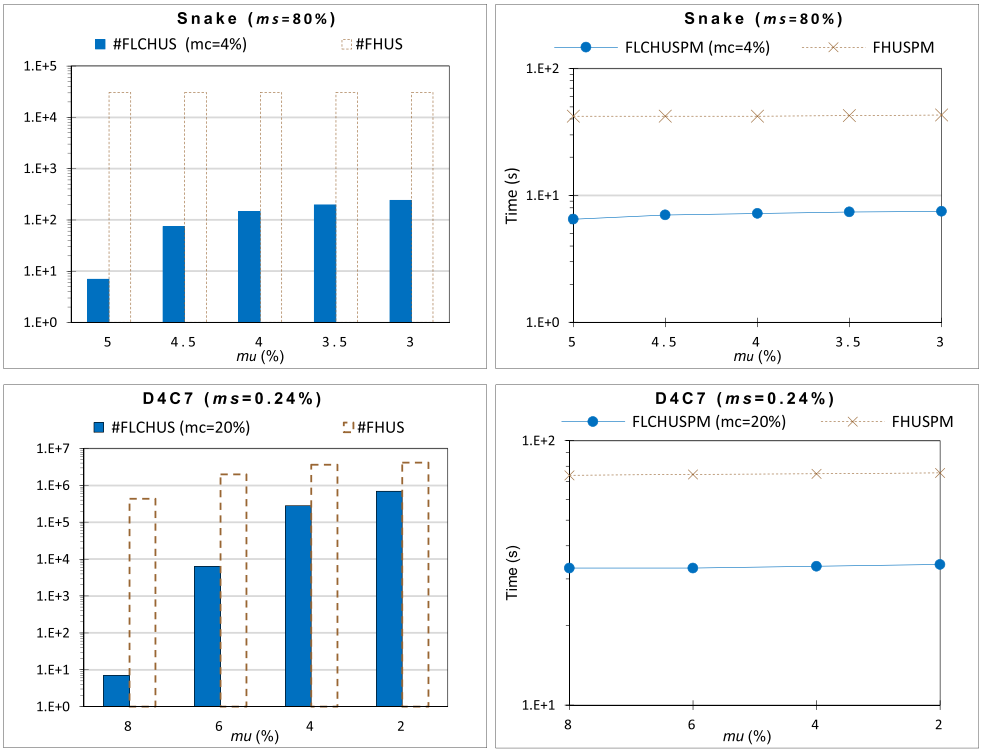


Fig. 9 Comparison of the number of mined patterns and time by FLCHUSPM and FHUSPM

Table 5 Comparison of patterns found by FLCHUSPM and FHUSPM

No	Sequential pattern	*au*	*ac*	*supp*
1	*Deeds_Es_5_3→Study_Materials*	10	24	1
	Deeds_Es_5_2→Study_Es_5_3→Deeds_Es_5_3→Deeds_Es_5_1	10	111	1
2	*Deeds_Es_5_4→Deeds_Es_5_1*	8.5	36.5	2
	Study_Es_5_2→Deeds_Es_5_4→Study_Es_5_4→Study_Es_5_4	8.5	67.5	2
3	*Deeds_Es_5_3→Deeds_Es_5_1*	5.67	32.33	3
	Deeds_Es_5_1→Deeds_Es_5_1→Deeds_Es_5_3→Deeds_Es_5_3→Deeds_Es_5_4	5.67	161.67	3
4	*Study_Es_5_3→Deeds_Es_5_1*	5.5	24	2
	Deeds_Es_5_1→Deeds_Es_5_4→Deeds_Es_5_4→Deeds_Es_5_4	5.5	100.25	4
5	*Study_Materials*	5	6	1
6	*Deeds_Es_5_2→Study_Es_5_4→Study_Es_5_4*	5	36.43	7
	Deeds_Es_5_1→Study_Es_5_1→Deeds_Es_5_3→Deeds_Es_5_4→Study_Es_5_4→Study_Es_5_4	5	195	2

parameters described in Table 2. For this experiment, the algorithm's thresholds were set to *ms*=0.95%, *mu*=0.005% and *mc*=0.1, and the dataset parameters N, C, D and T were varied one by one while others remained fixed. In Fig. 10, the wildcard (*) symbol is utilized in the name of the QSDB to indicate that a dataset parameter is varied. Results depicted in Fig. 10 show that when database parameters are increased, the memory requirement and execution time often also increase but not much.

Overall, all experiments of Section 6 have shown that FLCHUSPM largely outperforms the baseline algorithm thanks to the new *AMUB* and *FUB* UBs and the *FLB* LB, as well as the corresponding $\mathcal{WPS}$, $\mathcal{DPS}$ pruning, $\mathcal{RedS}$ reducing and *CMAPS* tightening strategies for efficiently pruning the search space. The performance of the proposed algorithm was evaluated both in terms of execution time and memory usage.

7 Conclusion

This paper presented the *FLCHUSM* problem of mining frequent sequences having a low average cost and high average utility. Depth and width pruning, tightening and reducing strategies were designed to solve the problem efficiently. These strategies rely on the anti-monotone-like and monotone-like properties of two new upper bounds (*AMUB* and *FUB*) on the average utility and a lower bound (*Flb*) on the average cost. They allow pruning infrequent or low utility or high-cost candidate branches of the prefix search tree early, reducing the database, and tightening candidate item sets for extensions. The strategies are implemented in the novel FLCHUSPM algorithm to completely and efficiently discover FLCHUS, thanks to a novel *CUL* data structure. Results from the experimental assessment have shown that FLCHUSPM is efficient in terms of execution time and memory consumption.

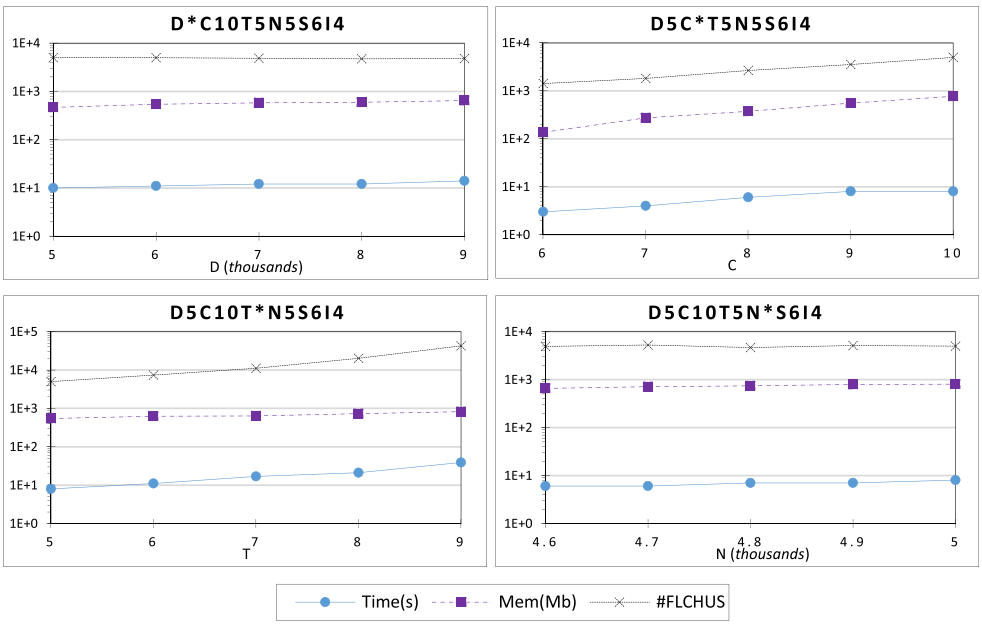


Fig. 10 Influence of database parameters on runtime, memory usage and number of patterns

For future work, the proposed problem and algorithm could be extended in different ways. First, an interesting research direction is to design methods to reduce redundancy in the patterns found, as several similar patterns may be discovered. An interesting solution to this problem is to integrate the idea of mining concise representations of patterns such as generator, closed and maximal patterns [14] in the proposed problem. This could be done by post-processing but would not be efficient. Designing an efficient algorithm to mine concise representations of patterns is not trivial and this will thus be considered as an extension in future work. Second, another interesting possibility is to redefine the problem as that of skyline pattern mining or top-k pattern mining [7, 35]. In top-k pattern mining, the user can directly specify the number k of patterns to discover, which is more intuitive than using *mins*. Third, another interesting research direction is to design alternative algorithms to further improve performance. Besides, we are also interested in designing a general method to quickly design upper and lower bounds for pattern mining problems, which is designed for anti-monotone or monotone-like properties of general interestingness measures.

References

1. Agrawal R, Srikant R (1995) Mining sequential patterns. In Proceedings of the Eleventh International Conference on Data Engineering, pp.3–14
2. Fournier-Viger P, Gomariz A, Campos M (2014) Fast vertical mining of sequential patterns using co-occurrence information. In Proceedings of 18th Pacific-Asia Conference on Knowledge Discovery and Data Mining, PAKDD '2014, pp.40–52
3. Wu R, Li Q, Chen X (2019) Mining contrast sequential pattern based on subsequence time distribution variation with discreteness constraints. Appl Intell 49(12):4348–4360
4. Ahmed CF, Tanbeer SK, Jeong BS (2010) Mining high utility web access sequences in dynamic web log data. In Proceedings of 11th ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing, SNPD2010, pp.76–81
5. Ryang H, Yun U (2016) High utility pattern mining over data streams with sliding window technique. Expert Syst Appl 57: 214–231
6. Tseng VS, Shie BE, Wu CW, Yu PS (2013) Efficient algorithms for mining high utility itemsets from transactional databases. IEEE Trans Knowl Data Eng 25(8):1772–1786
7. Zihayat M, Davoudi H, An A (2017) Top-k utility-based gene regulation sequential pattern discovery. In Proceedings of 2016 IEEE International Conference on Bioinformatics and Biomedicine, BIBM 2016, pp.266–273
8. Dalmas B, Fournier-Viger P, Norre S (2017) TWINCLE: a constrained sequential rule mining algorithm for event logs. In Proceedings of 9th International KES Conference (IDT-KES 2017), pp.205–214
9. Baek Y, Yun U, Kim H, Kim J, Vo B, Truong T (2021) Approximate high utility itemset mining in noisy environments. Knowledge-Based Syst 212:106596
10. Chan R, Yang Q, Shen Y-D (2003) Minging high utility itemsets. In Proceedings of IEEE International Conference on Data Mining, pp.19–26
11. Fournier-Viger P, Lin JC-W, Truong T, Nkambou R (2019) A survey of high utility Itemset mining. In High-Utility Pattern Mining: Theory, Algorithms and Applications; Fournier-Viger, Philippe; Jerry Chun-Wei., Lin; Nikambou, Roger; Vo, Bay; Tseng, Vincent S, Springer International Publishing. pp.1–44
12. Yin J, Zheng Z, Cao L (2012) USpan: An efficient algorithm for mining high utility sequential patterns. In Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp.660–668
13. Gan W, Lin JC-W, Zhang J, Fournier-Viger P, Chao H, Yu PS (2019) Fast utility mining on complex sequences. CoRR 1904(2): 1–15
14. Truong T, Duong H, Le B, Fournier-Viger P (2019) FMaxCloHUSM: An efficient algorithm for mining frequent closed and maximal high utility sequences. Eng Appl Artif Intell 85(1):1–20
15. Zhang J, Wang Y, Yang D (2015) CCSpan: mining closed contiguous sequential patterns. Knowledge-Based Syst 89(1):1–13
16. Zhang J, Wang Y, Zhang C, Shi Y (2016) Mining contiguous sequential generators in biological sequences. IEEE/ACM Trans Comput Biol Bioinforma 13(5):855–867
17. Truong T, Duong H, Le B, Fournier-Viger P, Yun U, Fujita H (2021) Efficient algorithms for mining frequent high utility sequences with constraints. Inf Sci (Ny) 568:239–264
18. Nguyen LTT, Vu VV, Lam MTH, Duong TTM, Manh LT, Nguyen TTT et al (2019) An efficient method for mining high utility closed itemsets. Inf Sci (Ny) 495:78–99
19. Hong T-P, Lee CH, Wang SL (2011) Effective utility mining with the measure of average utility. Expert Syst Appl 38(7):8259–8265
20. Yun U, Kim D (2017) Mining of high average-utility itemsets using novel list structure and pruning strategy. Futur Gener Comput Syst 68(1):346–360
21. Truong T, Duong H, Le B, Fournier-Viger P (2018) Efficient vertical Mining of High Average-Utility Itemsets Based on novel upper-bounds. IEEE Trans Knowl Data Eng 31(2):301–314
22. Truong T, Duong H, Le B, Fournier-Viger P, Yun U (2019) Efficient high average-utility itemset mining using novel vertical weak upper-bounds. Knowledge-Based Syst. 183(1):104847
23. Truong T, Duong H, Le B, Fournier-Viger P (2020) EHAUSM: An efficient algorithm for high average utility sequence mining. Inf Sci (Ny) 515(1):302–323
24. Yun U, Kim D, Yoon E, Fujita H (2018) Damped window based high average utility pattern mining over data streams. Knowledge-Based Syst 144:188–205
25. Zida S, Fournier-Viger P, Lin JC-W, Wu C-W, Tseng VS (2015) EFIM: a highly efficient algorithm for high-utility itemset mining. In Proceedings of Mexican International Conference on Artificial Intelligence (MICAI 2015), pp.530–546
26. Fournier-viger P, Zhang Y, Lin JC, Fujita H, Koh YS (2019) Mining local and peak high utility itemsets. Inf Sci (Ny). 481 344–367
27. Kim H, Yun U, Baek Y, Kim J, Vo B, Yoon E, et al. (2021) Efficient list based mining of high average utility patterns with maximum average pruning strategies. Inf Sci (Ny). 543 85–105
28. Ahmed CF, Tanbeer SK, Jeong BS (2010) A novel approach for mining high-utility sequential patterns in sequence databases. ETRI 32(5):676–686
29. Truong T, Tran A, Duong H, Le B, Fournier-Viger P (2020) EHUSM : mining high utility sequences with a pessimistic utility model. Data Sci Pattern Recognit 4(2):65–83
30. Alkan OK, Karagoz P (2015) CRoM and HuspExt: improving efficiency of high utility sequential pattern extraction. IEEE Trans Knowl Data Eng 27(10):2645–2657

31. Wang JZ, Huang JL, Chen YC (2016) On efficiently mining high utility sequential patterns. Knowl Inf Syst 49(2):597–627
32. Gan W, Lin JCW, Fournier-Viger P, Chao HC, Fujita H (2018) Extracting non-redundant correlated purchase behaviors by utility measure. Knowledge-Based Syst 143:30–41
33. Gan W, Lin JC, Zhang J, Chao H, Fujita H, Yu PS (2020) ProUM : projection-based utility mining on sequence data. Inf Sci (Ny). 513 222–240
34. Gan W, Lin JC, Chao H, Fujita H, Yu PS (2019) Correlated utility-based pattern mining. Inf Sci (Ny). 504 470–486
35. Yin J, Zheng Z, Cao L, Song Y, Wei W (2013) Efficiently mining top-K high utility sequential patterns. In Proceedings of 2013 IEEE 13th International Conference on Data Mining (ICDM), pp.1259–1264
36. Truong T, Fournier-Viger P (2019) A survey of high utility sequential pattern mining. In P. Fournier-Viger, J. C.-W. Lin, R. Nkambou, V. Bay, & V. S. Tseng, High-utility pattern mining: theory, algorithms and applications, pp.97–129
37. Thilagu M, Nadarajan R (2012) Efficiently mining of effective web traversal patterns with average utility. Procedia Technol 6(1):444–451
38. Lin JC-W, Li T, Pirouz M, Zhang J, Fournier-Viger P (2020) High average-utility sequential pattern mining based on uncertain databases. Knowl Inf Syst 62(3):1199–1228
39. Fournier-Viger P, Li J, Lin JC-W, Truong T (2019) Discovering and visualizing efficient patterns in cost/utility sequences. In Proceedings of International Conference on Big Data Analytics and Knowledge Discovery (DaWaK 2019), LNCS 11708, pp.73–88
40. Fournier-Viger P, Li J, Lin JC-W, Truong T, Kiran RU (2020) Mining cost-effective patterns in event logs Knowledge-Based Syst 191:105241
41. Fournier-Viger P, Lin JC-W, Gomaris A, Gueniche T, Soltani A, Deng Z et al (2014) SPMF: a Java open-source pattern mining library version 2. Mach Learn Res 15(1):3389–3393
42. Hong T-P, Lee CH, Wang SL (2009) Mining high average-utility itemsets. In Proceedings of IEEE International Conference on Systems, Man and Cybernetics, pp 2526–2530
43. Mehrnoosh V, Luca O, Davide A, Mathias F, Matthias R (2015) A learning analytics approach to correlate the academic achievements of students with interaction data from an educational simulator. In Lecture Notes in Computer Science, pp.613–616

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Tin Truong is a professor at the Department of Mathematics and Computer Science, University of Dalat, Vietnam. He received his B.S. degree in Mathematics from Dalat University in 1983 and his Ph.D. in Stochastic Optimal Control in 1990 at the Vietnam National University, VNU-Hanoi, Vietnam. His current research interests are artificial intelligence and data mining.



Hai Duong received his M.S. and Ph.D. degrees in Computer Science in 2009 and 2020 at the University of Science—VNU-HCMC, Vietnam. He is currently a lecturer and a researcher at the Department of Mathematics and Computer Science, Dalat University of Dalat, Vietnam. His research interest is data mining.



Bac Le received his B.S. degree in Mathematics in 1984 and the M.S. degree in Computer Sciences in 1990. He received his Ph.D. in Mathematics for Computers and Computing Systems in 2000 at the University of Science—VNU-HCMC, Vietnam. He is currently the Head of Department of Computer Science, Faculty of Information Technology. His current research interests include artificial intelligence, soft computing, data mining and data science.



Philippe Fournier-Viger is full professor at the Harbin Institute of Technology(Shenzhen). He has published more than 300 research papers in international conference proceedings and journals. He is associate editor-in-chief of Applied Intelligence and editor-in-chief of the Data Mining and Pattern Recognition journal. He is the founder of the popular SPMF open-source data mining library(http://www.philippe-fournier-viger.com/spmf/), which has been used in more than 1000 research papers since 2010. He is co-organizers of the UDML series of workshop at the KDD and ICDM conferences.



Unil Yun received the M.S. degree in computer science and engineering from Korea University, Seoul, South Korea, in 1997, and the Ph.D. degree in computer science from Texas A&M University, College Station, TX, USA, in 2005. He was with the Multimedia Laboratory, Korea Telecom, from 1997 to 2002. After receiving the Ph.D. degree, he was a Postdoctoral Associate for almost one year with the Computer Science Department, Texas A&M University. After that, he was a Senior Researcher with the Electronics and Telecommunications Research Institute. In March 2007, he joined the School of Electrical and Computer Engineering, Chungbuk National University, South Korea. Since August 2013, he has been with the Department of Computer Engineering, Sejong University, Seoul. His research interests include data mining, information retrieval, database systems, artificial intelligence, and digital libraries.